

INTRODUCCIÓN

CONTEXTO DEL PROYECTO

En la actualidad, la mayoría de las organizaciones, independientemente de su tamaño, gestionan grandes volúmenes de información en formato digital. Documentación técnica, manuales, informes internos, contratos, normativa, actas o históricos de proyectos se almacenan de forma continuada a lo largo del tiempo, generando repositorios documentales que crecen sin una estrategia clara de organización y explotación del conocimiento.

Esta situación es especialmente habitual en pequeñas y medianas empresas, donde los recursos técnicos y organizativos suelen ser limitados. En estos momentos, la gestión documental suele resolverse mediante estructuras de carpetas compartidas, sistemas de almacenamiento en red o soluciones genéricas en la nube, sin un modelo de datos definido ni mecanismos avanzados de búsqueda. Algo similar ocurre en departamentos técnicos o equipos reducidos dentro de organizaciones más grandes, donde la prioridad suele centrarse en la entrega de resultados inmediatos, relegando la gestión de la información a un segundo plano.

La ausencia de un gobierno del dato claro en estos contextos provoca consecuencias prácticas relevantes. La localización de información depende en gran medida del conocimiento previo de las personas que han participado en su creación o mantenimiento. Esto genera una dependencia excesiva de determinados perfiles clave, incrementando el riesgo operativo ante ausencias, rotaciones de personal o cambios organizativos. Además, la falta de herramientas adecuadas dificulta la reutilización de información existente, obligando en muchos casos a duplicar esfuerzos o a tomar decisiones basadas en información incompleta.

Desde el punto de vista operativo, esta situación se traduce en una pérdida de productividad, ya que tareas aparentemente simples, como localizar un documento o verificar una información concreta, pueden requerir un tiempo considerable. A ello se le suma el riesgo de errores derivados del uso de versiones incorrectas de documentos o de información desactualizada, con el impacto que ello puede tener en procesos técnicos, administrativos o incluso legales.

En paralelo, en los últimos años se ha producido un avance significativo en tecnologías relacionadas con el procesamiento de lenguaje natural y la inteligencia artificial aplicada a la información textual. Estas tecnologías permiten ir más allá de la búsqueda tradicional basada en palabras clave, ofreciendo mecanismos de acceso semántico al contenido de los documentos. Sin embargo, muchas de estas soluciones se presentan como herramientas cerradas, poco transparentes o difíciles de adaptar a contextos concretos, lo que limita su adopción en entornos pequeños o académicos.

Este Proyecto de Fin de Ciclo se enmarca en este contexto, planteando el desarrollo de una solución que parte de una gestión documental sólida y estructurada como base para aplicar, de forma progresiva y controlada, técnicas de acceso inteligente a la información.

MOTIVACIÓN

La motivación principal para la realización de este proyecto puede analizarse desde tres perspectivas complementarias: profesional, académica y personal-técnica.

Desde el punto de vista profesional, el proyecto responde a una problemática real observada en múltiples contextos laborales y formativos: la dificultad para gestionar y explotar de forma eficiente la información documental. En muchos entornos, la tecnología existe, pero se aplica de manera fragmentada o sin una visión global del sistema. Este proyecto busca reproducir una situación cercana a la realidad profesional, donde es necesario analizar un problema, acotar su alcance y proponer una solución técnica viable, teniendo en cuenta limitaciones de tiempo y recursos.

En el ámbito académico, el proyecto supone una oportunidad para integrar de forma práctica los conocimientos adquiridos a lo largo del ciclo de Desarrollo de Aplicaciones Multiplataforma. A diferencia de ejercicios aislados o prácticas guiadas, el Proyecto de Fin de Ciclo exige una visión global del sistema, así como la capacidad de planificar, documentar y justificar cada decisión tomada. Además, el proyecto requiere ampliar estos conocimientos mediante el aprendizaje autónomo, profundizando en tecnologías y conceptos que no se abordan de forma exhaustiva durante el ciclo formativo.

Por último, existe una modalidad personal y técnica vinculada al interés por la aplicación responsable de técnicas de inteligencia artificial. Frente a enfoques que

priorizan resultados espectaculares sin un control claro del proceso, este proyecto apuesta por una aproximación gradual, donde la inteligencia artificial se apoya en una base sólida de gestión de la información y trazabilidad. El objetivo no es demostrar el uso de una tecnología concreta, sino comprender cómo y cuándo aporta valor real dentro de un sistema software.

OBJETIVOS DEL DOCUMENTO

El objetivo principal de este documento es presentar de manera clara, estructurada y evaluable el desarrollo del proyecto realizado. La memoria no se limita a describir el resultado final sino que documenta el proceso seguido, las decisiones adoptadas y las limitaciones asumidas.

Desde un punto de vista técnico, el documento tiene como objetivo describir la arquitectura del sistema, las funcionalidades implementadas y la forma en que se integran los distintos componentes. Cada bloque funcional se presenta de manera que pueda ser evaluado de forma objetiva, apoyándose en evidencias y criterios de validación definidos previamente.

Desde una perspectiva metodológica, la memoria tiene como finalidad demostrar que el proyecto ha sido planificado y ejecutado siguiendo una metodología coherente, con una gestión clara de requisitos, tiempos y riesgos. La trazabilidad entre los requisitos definidos, las tareas realizadas y los resultados obtenidos es un aspecto central del documento.

Finalmente, el documento persigue un objetivo formativo: evidenciar el aprendizaje adquirido durante el desarrollo del proyecto, tanto a partir de los contenidos del ciclo como del aprendizaje autónomo necesario para abordar un sistema de esta complejidad.

ESTRUCTURA DE LA MEMORIA

La memoria se estructura en varios capítulos que reflejan la evolución lógica del proyecto. Tras esta introducción, se presenta la definición del proyecto y los requisitos iniciales, donde se detalla el problema a resolver, los objetivos y el alcance. Posteriormente, se describe la metodología de trabajo y la planificación adoptada.

Los capítulos siguientes abordan el análisis de riesgos, la arquitectura del sistema y el desarrollo de las distintas funcionalidades. A continuación, se presenta la integración de técnicas de inteligencia artificial, la validación del sistema y el seguimiento del proyecto. Finalmente, se exponen las conclusiones y posibles líneas de trabajo futuro.

DEFINICIÓN Y REQUISITOS INICIALES (R01-R02)

DESCRIPCIÓN DEL PROBLEMA A RESOLVER (R01)

La gestión documental tradicional se basa, en la mayoría de los casos, en estructuras jerárquicas de carpetas y en búsquedas por nombre de archivo o metadatos básicos. Este enfoque resulta ineficiente cuando el volumen de documentos crece o cuando el contenido de los mismos es complejo y heterogéneo.

Las búsquedas clásicas presentan limitaciones evidentes: requieren conocer de antemano términos concretos, no tienen en cuenta el contexto semántico y ofrecen resultados poco relevantes cuando el lenguaje utilizado en los documentos no coincide exactamente con la consulta. Aunque el uso de metadatos mejora parcialmente esta situación, su mantenimiento suele ser manual y dependiente del criterio de los usuarios, lo que introduce inconsistencias y limita su eficacia.

El problema, por tanto, no es únicamente tecnológico, sino también estructural. Sin una base sólida de gestión documental, cualquier intento de aplicar mecanismos avanzados de búsqueda o generación de respuestas se vuelve frágil y difícil de justificar. Este proyecto aborda el problema desde esa raíz: establecer primero un sistema coherente de gestión de documentos y, a partir de él, explorar nuevas formas de acceso a la información.

OBJETIVOS GENERALES Y ESPECÍFICOS (R01)

El objetivo general del proyecto es desarrollar un sistema software que permita gestionar documentación de forma estructurada y facilitar un acceso inteligente a su contenido.

Para alcanzar este objetivo general, se definen varios objetivos específicos, cada uno de los cuales responde a una necesidad concreta del sistema. En primer lugar, se plantea la

implementación de un sistema de gestión de usuarios y control de acceso, como base para garantizar la seguridad y la trazabilidad de las acciones realizadas. A continuación, se aborda la gestión documental propiamente dicha, incluyendo almacenamiento, versionado y metadata.

Una vez establecida esta base, el proyecto se centra en el procesamiento del contenido de los documentos, con el objetivo de extraer información textual y prepararla para su posterior explotación. Sobre esta información procesada se implementan mecanismos de búsqueda semántica y, finalmente, un flujo básico de generación de respuestas a partir del contenido documental.

Estos objetivos presentan dependencias claras entre sí, lo que refuerza la necesidad de una planificación cuidadosa y de una ejecución ordenada del proyecto.

ALCANCE DEL PROYECTO (R02)

El alcance del proyecto incluye el diseño y el desarrollo de un sistema backend orientado a la gestión documental y a la recuperación de información. Cada módulo del sistema se aborda de forma progresiva, comenzando por los elementos básicos y avanzando hacia funcionalidades más complejas.

El sistema incluye funcionalidades de autenticación, control de roles, almacenamiento de documentos, gestión de versiones, asociación de metadata y procesamiento del contenido textual. Asimismo, se contempla la implementación de mecanismos de búsqueda semántica y un flujo básico de generación de respuestas apoyado en técnicas de inteligencia artificial.

Cada una de estas funcionalidades se desarrolla en una fase concreta del proyecto, lo que permite controlar la complejidad y evaluar el progreso de forma objetiva. El alcance incluye también la documentación, validación y despliegue básico del sistema en un entorno controlado.

EXCLUSIONES Y SUPUESTOS (R02)

Con el fin de mantener el proyecto realista y acorde al nivel académico del ciclo, se establecen una serie de exclusiones explícitas. No se incluyen optimizaciones avanzadas

de rendimiento, despliegue en entornos productivos reales ni comparativas exhaustivas entre distintos modelos de inteligencia artificial.

Estas exclusiones se justifican por limitaciones de tiempo, recursos y alcance académico. El objetivo del proyecto no es desarrollar un producto comercial, sino demostrar la capacidad de diseñar y construir un sistema coherente, funcional y evaluable.

Asimismo, se asume que el sistema se ejecuta en un entorno controlado y que los usuarios disponen de conocimientos básicos para interactuar con él. Estos supuestos permiten centrar el esfuerzo en los aspectos clave del proyecto y evitar desviaciones innecesarias.

METODOLOGÍA Y PLANIFICACIÓN (R04)

Este capítulo describe la metodología del trabajo adoptada para el desarrollo del proyecto y la planificación inicial realizada antes del inicio de la fase de implementación. El objetivo principal es justificar de forma clara y estructurada el enfoque seguido, demostrando que el proyecto ha sido concebido y organizado de manera coherente, trazable y acorde a un contexto académico y técnico.

La planificación presentada en este capítulo corresponde exclusivamente a la estimación inicial del proyecto. El análisis de la ejecución real, así como las posibles desviaciones respecto a dicha planificación, se aborda en un capítulo posterior dedicado al seguimiento del proyecto.

METODOLOGÍA DE TRABAJO ADOPTADA

El desarrollo del proyecto se ha planteado siguiendo un enfoque iterativo e incremental, estructurando en torno a requisitos claramente definidos. En lugar de abordar el sistema como un bloque monolítico, el proyecto se divide en unidades de trabajo coherentes que permiten avanzar de forma progresiva y controlada.

Cada requisito representa una unidad funcional completa, con un objetivo concreto, un alcance delimitado y unos criterios de finalización explícitos. Este enfoque permite reducir la complejidad del desarrollo, ya que cada bloque puede analizarse, planificarse

y validarse de forma independiente, manteniendo al mismo tiempo una visión global del sistema.

La metodología adoptada prioriza la trazabilidad y el control frente a la rapidez de ejecución. En un contexto académico, resulta fundamental poder justificar no solo qué ha desarrollado, sino también por qué se han tomado determinadas decisiones y cómo se ha organizado el trabajo. Por este motivo, se ha optado por un modelo que facilita la documentación continua del proceso y la evaluación objetiva de cada fase del proyecto.

Además, este enfoque permite adaptar el ritmo de trabajo a la complejidad real de cada requisito, evitando la imposición de estructuras temporales rígidas que no siempre se ajustan a la naturaleza del desarrollo individual. La metodología seleccionada busca, por tanto, un equilibrio entre orden, flexibilidad y capacidad de seguimiento.

JUSTIFICACIÓN DEL USO DE KANBAN

Para la gestión del trabajo se ha optado por el uso de Kanban como metodología de organización y seguimiento. Esta elección responde a las características específicas del proyecto y a su contexto de desarrollo individual.

Kanban permite visualizar de forma clara el estado de cada requisito mediante un flujo sencillo compuesto por distintas columnas que representan las fases de trabajo. En este proyecto, dicho flujo se estructura en estados como backlog, tareas pendientes, trabajos en curso, revisión y finalización. Esta visualización continua facilita el control de avance y la identificación temprana de bloqueos o dependencias.

A diferencia de metodologías basadas en iteraciones temporales cerradas, Kanban no impone ciclos fijos ni compromisos de entrega por *sprint*. Esta flexibilidad resulta especialmente adecuada para un proyecto individual, donde la carga de trabajo puede variar significativamente entre requisitos y donde no siempre es posible predecir con exactitud la duración de cada tarea desde el inicio.

Otro aspecto relevante es que Kanban favorece la gestión del trabajo basada en el flujo, permitiendo centrarse en completar requisitos de forma progresiva y ordenada. Este enfoque resulta coherente con el objetivo del proyecto, que prioriza la calidad y la trazabilidad del desarrollo frente a la velocidad de ejecución.

Por último, el uso de Kanban facilita la integración con herramientas de control de versiones y repositorios de código, permitiendo asociar de forma directa cada requisito con sus evidencias de desarrollo y documentación. Esto refuerza la transparencia del proceso y mejora la capacidad de evaluación del proyecto.

MODELO RFTP APLICADO AL PROYECTO

Con el objetivo de estructurar el trabajo de forma clara y trazable, el proyecto adopta el modelo RFTP, que organiza el desarrollo en torno a cuatro niveles: Requisitos (R), Funciones (F), Tareas (T) y Pruebas (P).

En este proyecto, los requisitos constituyen la unidad principal de planificación y seguimiento. Cada requisito define una funcionalidad o bloque de trabajo claramente identificable, alineado con los objetivos del sistema. Los requisitos son los únicos elementos gestionados directamente en el tablero Kanban, lo que permite mantener una visión de alto nivel sin fragmentar la planificación.

Dentro de cada requisito, se identifican las funciones que debe cumplir el sistema, las tareas técnicas necesarias para su implementación y las pruebas asociadas para su validación. Esta descomposición interna no se refleja como elementos independientes en el Kanban, sino que se documenta dentro de cada requisito, evitando así una sobrecarga de gestión innecesaria.

El modelo RFTP permite esclarecer una trazabilidad clara entre los objetivos definidos inicialmente y las evidencias generadas durante el desarrollo del proyecto. Cada requisito puede vincularse con commits, pruebas y documentación, facilitando la justificación del trabajo realizado y la evaluación del cumplimiento de los objetivos planteados.

Este enfoque resulta especialmente adecuado en un contexto académico, ya que combina un alto nivel de control con una estructura comprensible y defendible.

Aplicación práctica del modelo RFTP en el proyecto

Una vez definido el modelo RFTP como marco metodológico, se procede a aplicarlo de forma sistemática a los requisitos del proyecto. No todos los requisitos presentan el mismo nivel de descomposición: aquellos de carácter conceptual o documental (R01–

R05) se estructuran principalmente a nivel de requisito, mientras que los bloques funcionales y técnicos (R10 en adelante) se desglosan en funciones, tareas y pruebas verificables.

Esta aplicación permite mantener coherencia entre planificación, implementación y validación. Cada requisito técnico implementado en el backend, el sistema de búsqueda o el flujo RAG dispone de tareas identificables y pruebas asociadas que garantizan su correcto funcionamiento.

A continuación, se presenta la descomposición estructurada de los requisitos principales del proyecto según el modelo RFTP.

R01 – Definición del problema y objetivos

R02 – Alcance, exclusiones y supuestos

R03 – Arquitectura global del sistema

R04 – Metodología Kanban y planificación

R05 – Identificación de riesgos

R10 – Modelo de usuarios

- **F10** – Gestión básica de usuarios
 - **T10.1** – Entidad usuario definida y mapeada
 - **T10.2** – Repositorio de acceso a datos
 - **P10.1** – Prueba de persistencia y recuperación

R11 – Autenticación JWT + refresh tokens

- **F11** – Autenticación de usuarios
 - **T11.1** – Generación de access token JWT
 - **T11.2** – Validación de token en peticiones protegidas
 - **T11.3** – Implementación de refresh token
 - **P11.1** – Prueba de login y acceso protegidos
 - **P11.2** – Prueba de renovación de token

R12 – RBAC + Permisos por documento (ABAC/ACL simple)

- **F12** – Gestión de roles
 - **T12.1** – Asociación usuario-rol
 - **T12.2** – Restricción de acceso por rol (RBAC)
 - **T12.3** – Control de acceso por documento (ACL simple)

- **P12.1** – Prueba de acceso autorizado / no autorizado
- **P12.2** – Prueba de acceso a documento según ownership

R13 – Test de seguridad

- **P13.1** – Prueba de login válido / inválido
- **P13.2** – Prueba de acceso con token válido
- **P13.3** – Prueba de acceso con token inválido o expirado
- **P13.4** – Prueba de renovación mediante refresh token
- **P13.5** – Prueba de acceso sin permisos suficientes
- **P13.6** – Prueba de acceso a documento sin ownership

R14 – Rate limiting en endpoints críticos

- **F14** – Protección de endpoints críticos
 - **T14.1** – Middleware de limitación de tasa
 - **T14.2** – Integración con Redis
 - **P14.1** – Pruebas de límite superado (HTTP 429)

R15 – Observabilidad y monitorización del sistema

- **F15** – Generación de logs estructurados
 - **T15.1** – Middleware de contexto de petición
 - **T15.2** – Implementación de métricas Prometheus
 - **P15.1** – Verificación de logs y endpoints / metrics

R20 – Subida de documentos

- **F20** – Subida de documento
 - **T20.1** – Endpoint REST de subida
 - **T20.2** – Persistencia del archivo y referencia
 - **P20.1** – Prueba de subida correcta

R21 – Metadata de documentos

- **F21** – Gestión del metadata
 - **T21.1** – Modelo de metadata
 - **T21.2** – Asociación metadata-documento
 - **P21.1** – Prueba de gestión del metadata

R22 – Versionado de documentos

- **F22** – Gestión de versiones
 - **T22.1** – Modelo de versión

- **T22.2** – Lógica de creación de versiones
- **P22.1** – Prueba de versionado

R23 – Test de gestión documental

- **P23.1** – Prueba integrada de subida + metadata
- **P23.2** – Prueba de versionado sobre documento existente
- **P23.3** – Prueba de coherencia entre documento, metadata y versiones

R30 – Extracción de texto

- **F30** – Extracción de texto del documento
- **T30.1** – Servicio de extracción implementado
- **T30.2** – Persistencia del texto extraído
- **P30.1** – Prueba de extracción correcta

R31 – Chunking

- **F31** – Fragmentación del texto
- **T31.1** – Lógica de chunking
- **T31.2** – Persistencia de chunks
- **P31.1** – Prueba chunking correcto

R32 – Pipeline asíncrono

- **F32** – Orquestación del pipeline
- **T32.1** – Ejecución asíncrona
- **T32.2** – Gestión de estados / errores
- **P32.1** – Prueba de ejecución completa del pipeline

R33 – Test del pipeline

- **P33.1** – Prueba end-to-end: documento procesado completamente
- **P33.2** – Verificación de estados del pipeline
- **P33.3** – Validación de coherencia entre texto extraído, chunks y estados

R40 – Generación de embeddings

- **F40** – Generación de embeddings
- **T40.1** – Integración con modelo de embeddings
- **T40.2** – Asociación embedding-chunk
- **P40.1** – Prueba de generación de embeddings

R41 – Almacenamiento pgvector

- **F41** – Persistencia de embeddings

- **T41.1** – Configuración pgvector
- **T41.2** – Mapeo de columnas vectoriales
- **P41.1** – Prueba de almacenamiento vectorial

R42 – Índices vectoriales

- **F42** – Índices vectoriales activos
- **T42.1** – Configuración de métricas de similitud
- **P42.1** – Prueba de consulta vectorial

R43 – Tests vectoriales

- **P43.1** – Prueba end-to-end: chunk → embedding → persistencia en pgvector → recuperación por similitud
- **P43.2** – Verificación de coherencia: el embedding recuperado corresponde al chunk esperado
- **P43.3** – Validación de índice: consulta vectorial eficiente / operativa con índice configurado

R50 – Búsqueda semántica básica

- **F50** – Consulta semántica
- **T50.1** – Implementación de búsqueda vectorial
- **T50.2** – Recuperación de resultados
- **P50.1** – Prueba de búsqueda semántica

R51 – Búsqueda híbrida

- **F51** – Búsqueda híbrida
- **T51.1** – Integración texto + vector
- **T51.2** – Combinación de resultados
- **P51.1** – Prueba de búsqueda híbrida

R52 – Ranking explicable

- **F52** – Ranking de resultados
- **T52.1** – Cálculo de score de relevancia
- **T52.2** – Exposición de información explicable
- **P52.1** – Prueba de ranking

R53 – Filtros por metadata

- **F53** – Filtrado por metadata
- **T53.1** – Aplicación de filtros

- **P53.1** – Prueba de filtrado

R54 – Seguridad por documento en retrieval (filtro owner)

- **F54** – Seguridad por documento en búsqueda
 - **T54.1** – Aplicación de filtro owner en retrieval
 - **T54.2** – Integración con permisos definidos en R12
 - **P54.1** – Prueba de recuperación autorizada / no autorizada
 - **P54.2** – Prueba de exclusión de documentos no permitidos

R55 – Tests de búsquedas

- **P55.1** – Prueba end-to-end: consulta híbrida con ranking y filtros activos
- **P55.2** – Validación conjunta de ranking + metadata + seguridad
- **P55.3** – Verificación de exclusión de resultados no autorizados en escenario real
- **P55.4** – Comparación de resultados autorizados vs no autorizados en escenario real

R56 – Front básico de búsqueda

- **F56** – Interfaz de búsqueda
 - **T56.1** – Integración con backend
 - **P56.1** – Prueba de visualización de resultados en UI

R70 – Implementación RAG básica

- **F70** – Flujo RAG básico
 - **T70.1** – Recuperación de contexto
 - **T70.2** – Construcción del prompt
 - **P70.1** – Prueba de generación de respuesta

R71 – Control de alucinaciones

- **F71** – Control de alucinaciones
 - **T71.1** – Validación de contexto / respuesta
 - **P71.1** – Prueba de control de alucinaciones

R72 – Sistema de citas

- **F72** – Generación de citas
 - **T72.1** – Asociación chunk-respuesta
 - **P72.1** – Prueba de citas

R73 – Evaluación automática

- **F73** – Evaluación automática
 - **T73.1** – Cálculo de métricas básicas
 - **P73.1** – Prueba de evaluación

R74 – Métricas de latencia y coste

- **F74** – Métricas de latencia
 - **T74.1** – Registro de métricas de coste
 - **P74.1** – Pruebas de medición

R75 – Test IA

- **P75.1** – Prueba end-to-end: consulta → retrieval → generación → respuesta con citas
- **P75.2** – Validación conjunta de control de alucinaciones + citas
- **P75.3** – Verificación de registro de métricas durante ejecución completa

R80 – Tracking de eventos

- **F80** – Registro de eventos
 - **T80.1** – Modelo de eventos
 - **T80.2** – Persistencia de eventos
 - **P80.1** – Prueba de registro de eventos

R82 – Dashboard Power BI

- **F82** – Visualización de eventos
 - **T82.1** – Modelo analítico
 - **T82.2** – Creación de dashboard
 - **P82.1** – Prueba de visualización

R90 – Docker Compose

R91 – Script de despliegue

R92 – Manual de instalación

R93 – Manual de usuario

PLANIFICACIÓN INICIAL POR REQUISITOS

A partir de los requisitos definidos en las fases iniciales del proyecto, se ha elaborado una planificación inicial que organiza el trabajo por bloques funcionales y asigna una estimación temporal a cada uno de ellos. Esta planificación tiene como objetivo

proporcionar una visión global del esfuerzo previsto y servir como referencia para el seguimiento posterior del proyecto.

Los requisitos se agrupan en distintas fases que reflejan la evolución lógica del sistema, comenzando por tareas de análisis y planificación, y avanzando progresivamente hacia el desarrollo del núcleo del sistema, la integración de funcionalidades avanzadas y el cierre del proyecto. Esta organización facilita la comprensión del proyecto como un todo y permite identificar dependencias entre los distintos bloques de trabajo.

La planificación se ha realizado asignando una estimación de horas a cada requisito, teniendo en cuenta su complejidad técnica, el nivel de conocimiento previo y el esfuerzo necesario para su correcta documentación y validación. Estas estimaciones no pretenden ser exactas, sino realistas y coherentes con el alcance del proyecto.

Tabla 3.1 – Planificación inicial del proyecto por requisitos

La tabla 3.1 recoge la planificación inicial del proyecto, estructurada por fases y desglosada en requisitos. Para cada requisito se indica la estimación temporal prevista, así como su clasificación funcional mediante *labels*, lo que permite una visión global del esfuerzo estimado y de la distribución del trabajo a lo largo del proyecto.

FASE 0 – PREPARACIÓN

ID	Requisito	Horas	Labels
R01	Definición del problema y objetivos	6	documentación, fase-0-preparación
R02	Alcance, exclusiones y supuestos	4	documentación, fase-0-preparación
R03	Arquitectura global del sistema	8	arquitectura, fase-0-preparación, core
R04	Metodología Kanban y planificación	4	documentación, fase-0-preparación
R05	Identificación de riesgos	2	documentación, fase-0-preparación
Subtotal Fase 0		24 h	

FASE 1 - BACKEND CORE

Seguridad y Usuarios			
ID	Requisito	Horas	Labels
R10	Modelo de usuarios	7	desarrollo, fase-1-backend, core
R11	Autenticación JWT + Refresh Tokens	8	desarrollo, fase-1-backend, core
R12	RBAC + Permisos por documento (ABAC/ACL simple)	6	desarrollo, fase-1-backend, core
R13	Tests de seguridad	4	pruebas, fase-1-backend, core
R14	Rate limiting (login/query/upload)	6	desarrollo, fase-1-backend, core

Seguridad y Usuarios			
R15	Observabilidad (logs estructurados + métricas/metrics)	8	desarrollo, fase-1-backend, core
Subtotal		39 h	

Gestión Documental			
ID	Requisito	Horas	Labels
R20	Subida de documentos	8	desarrollo, fase-1-backend, core
R21	Metadata de documentos	6	desarrollo, fase-1-backend, core
R22	Versionado de documentos	6	desarrollo, fase-1-backend, core
R23	Tests gestión documental	4	pruebas, fase-1-backend, core
Subtotal		24 h	

Ingesta y procesamiento			
ID	Requisito	Horas	Labels
R30	Extracción de texto	8	desarrollo, fase-1-backend, core
R31	Chunking	6	desarrollo, fase-1-backend, core
R32	Pipeline asíncrono	8	desarrollo, fase-1-backend, core
R33	Tests pipeline	4	pruebas, fase-1-backend, core
Subtotal		26 h	

Vectorización			
ID	Requisito	Horas	Labels
R40	Generación de embeddings	8	desarrollo, fase-1-backend, ia
R41	Almacenamiento pgvector	6	desarrollo, fase-1-backend, infra
R42	Índices vectoriales	4	desarrollo, fase-1-backend, infra
R43	Tests vectoriales	4	pruebas, fase-1-backend, ia
Subtotal Fase 1		111 h	

FASE 2 - BÚSQUEDA SEMÁNTICA

ID	Requisito	Horas	Labels
R50	Búsqueda semántica básica	8	desarrollo, fase-2-búsqueda, ia
R51	Búsqueda híbrida	6	desarrollo, fase-2-búsqueda, ia
R52	Ranking explicable	6	desarrollo, fase-2-búsqueda, ia
R53	Filtros por metadata	4	desarrollo, fase-2-búsqueda, core
R54	Seguridad por documento en retrieval (filtro owner)	4	desarrollo, fase-2-búsqueda, core
R55	Tests de búsqueda	4	pruebas, fase-2-búsqueda, ia
R56	Front básico de búsqueda	6	desarrollo, fase-2-búsqueda, core
Subtotal Fase 2		38 h	

FASE 3 - RAG + IA

ID	Requisito	Horas	Labels
R70	Implementación RAG básica	10	desarrollo, fase-3-rag, ia
R71	Control de alucinaciones	6	desarrollo, fase-3-rag, ia
R72	Sistema de citas	4	desarrollo, fase-3-rag, ia
R73	Evaluación automática	6	desarrollo, fase-3-rag, ia
R74	Métricas latencia/coste	4	desarrollo, fase-3-rag, ia
R75	Tests IA	4	pruebas, fase-3-rag, ia
Subtotal Fase 3		34 h	

FASE 4 - DATA & ANALYTICS

ID	Requisito	Horas	Labels
R80	Tracking de eventos	6	desarrollo, fase-4-analytics, data
R82	Dashboard Power BI	8	desarrollo, fase-4-analytics, data
Subtotal Fase 4		14 h	

FASE 5 - CIERRE

ID	Requisito	Horas	Labels
R90	Docker Compose	6	cierre, fase-5-cierre, infra
R91	Script de despliegue	4	cierre, fase-5-cierre, infra
R92	Manual de instalación	4	cierre, fase-5-cierre, documentación
R93	Manual de usuario	4	cierre, fase-5-cierre, documentación
R94	Preparación defensa	4	cierre, fase-5-cierre, documentación
Subtotal Fase 5		22 h	

TOTALIZACIÓN FINAL

Concepto	Horas
Fase 0 - Preparación	24 h
Fase 1 - Backend core	111 h
Fase 2 - Búsqueda semántica	38 h
Fase 3 - RAG + IA	34 h
Fase 4 - Analytics	14 h
Fase 5 - Cierre	22 h
Total Proyecto	235 h

La planificación presentada corresponde a una estimación inicial realizada antes del inicio del desarrollo del proyecto. Esta estimación se utiliza como referencia para el seguimiento posterior del trabajo y el análisis de desviaciones, que se aborda en un capítulo específico dedicado al control y evaluación del proyecto.

ESTIMACIÓN TEMPORAL DEL PROYECTO (235 h)

La estimación temporal del proyecto asciende a 235 horas, distribuidas a lo largo de las distintas fases definidas en la planificación inicial. Esta estimación tiene en cuenta no solo el desarrollo técnico de las funcionalidades, sino también el tiempo necesario para la documentación y la validación del sistema.

La mayor carga de trabajo se concentra en las fases de desarrollo backend, la búsqueda semántica y la integración de componentes de inteligencia artificial. Estas fases presentan una mayor complejidad técnica y requieren un mayor esfuerzo de diseño, implementación y prueba. Por el contrario, las fases iniciales de análisis y las fases finales de cierre presentan una carga horaria menor, aunque igualmente relevante desde el punto de vista metodológico y académico.

La distribución temporal propuesta refleja un enfoque realista y acorde con los objetivos del proyecto y el nivel formativo del ciclo. La estimación inicial permite, además, establecer un marco de referencia claro para evaluar el progreso del proyecto y analizar de forma objetiva las desviaciones que puedan producirse durante su ejecución.

CIERRE DEL CAPÍTULO

El enfoque metodológico y la planificación descritos en este capítulo constituyen la base sobre la que se desarrolla el resto del proyecto. La combinación de Kanban como metodología de gestión y del modelo RFTP como estructura de trabajo permite abordar el proyecto de forma ordenada, controlada y trazable, facilitando tanto su desarrollo como su evaluación académica.

ANÁLISIS DE RIESGOS (R05)

El análisis de riesgos constituye una parte fundamental en la planificación de cualquier proyecto de software, ya que permite anticipar posibles problemas y definir medidas

para reducir su impacto. En un contexto académico, este análisis no persigue eliminar todos los riesgos, sino demostrar que el proyecto se ha planteado con una visión realista y consciente de sus limitaciones técnicas, organizativas y temporales.

En este proyecto, el análisis de riesgos se ha realizado teniendo en cuenta tanto los aspectos técnicos del sistema como el contexto organizativo de un desarrollo individual. Los riesgos identificados se evalúan en función de su probabilidad de ocurrencia y su impacto potencial sobre el desarrollo del proyecto, estableciendo medidas de mitigación coherentes y proporcionadas al alcance del trabajo.

IDENTIFICACIÓN DE RIESGOS

La identificación de riesgos se ha llevado a cabo analizando las distintas fases del proyecto y los elementos clave que lo componen. A partir de este análisis, se han identificado riesgos tanto de carácter técnico como organizativo.

Entre los riesgos técnicos, destaca la complejidad asociada a la integración de múltiples componentes dentro de un mismo sistema. El proyecto combina gestión documental, procesamiento de texto, almacenamiento vectorial y técnicas de búsqueda semántica, lo que implica la interacción de distintas tecnologías y bibliotecas. Esta integración puede dar lugar a problemas de compatibilidad, errores difíciles de aislar o comportamientos no previstos.

Otro riesgo técnico relevante es el aprendizaje de tecnologías no abordadas en profundidad durante el ciclo formativo. Algunas partes del proyecto requieren un proceso de aprendizaje autónomo que puede afectar a la estimación temporal inicial y al ritmo del desarrollo, especialmente en las fases más avanzadas del sistema.

Asimismo, existe el riesgo de que el procedimiento de documentos y la generación de información derivada no ofrezcan los resultados esperados en determinados casos, lo que podría limitar la funcionalidad del sistema o requerir ajustes adicionales.

Desde el punto de vista organizativo, el principal riesgo identificado es el desarrollo del proyecto por una única persona. Esta circunstancia implica una dependencia total de la disponibilidad del desarrollador y limita la capacidad de repartir tareas o revisar el trabajo de forma externa. A ello se suma la necesidad de compaginar el desarrollo del

proyecto con otras obligaciones académicas, lo que puede afectar a la planificación prevista.

Por último, se identifica como riesgo organizativo la posible desviación entre la planificación inicial y la ejecución real del proyecto, especialmente en aquellas fases con mayor carga técnica o incertidumbre.

IMPACTO Y PROBABILIDAD

Una vez identificados los riesgos principales, se ha procedido a evaluar su impacto y probabilidad con el fin de priorizar las medidas de mitigación. Esta evaluación se ha realizado de forma cualitativa, clasificando cada riesgo en niveles bajos, medios o altos.

Los riesgos asociados a la integración de componentes presentan una probabilidad media y un impacto medio-alto ya que pueden ofrecer retrasos en el desarrollo o requerir cambios en el diseño inicial. No obstante, su impacto se considera controlable mediante una planificación progresiva y una validación temprana de cada bloque funcional.

El riesgo derivado del aprendizaje autónomo presenta una probabilidad alta, dado que el proyecto incluye tecnologías que requieren estudio previo. Sin embargo, su impacto se considera medio, ya que forma parte del objetivo formativo del proyecto y puede gestionarse ajustando la planificación y priorizando funcionalidades esenciales.

En cuanto a los riesgos relacionados con el procesamiento de documentos y la calidad de los resultados obtenidos, su probabilidad se considera media, con un impacto moderado. Estos riesgos no comprometen el núcleo del sistema, pero pueden afectar a la calidad percibida de algunas funcionalidades avanzadas.

Los riesgos organizativos asociados al desarrollo individual presentan una probabilidad alta, ya que dependen directamente de la carga de trabajo personal. Su impacto puede ser significativo si no se gestionan adecuadamente, aunque se considera mitigable mediante una planificación realista y un seguimiento continuo del progreso.

Por último, la posible desviación entre planificación y ejecución se considera un riesgo con probabilidad media y un impacto controlable, siempre que se documente y analice de forma adecuada, tal y como se plantea en los capítulos posteriores de la memoria.

MEDIDAS DE MITIGACIÓN

Para cada uno de los riesgos identificados se han definido medidas de mitigación orientadas a reducir su probabilidad de ocurrencia o a minimizar su impacto sobre el proyecto. Estas medidas se han diseñado teniendo en cuenta el alcance académico del trabajo y los recursos disponibles.

En el caso de los riesgos técnicos relacionados con la integración de componentes, se ha optado por un desarrollo incremental, validando cada bloque funcional de forma independiente antes de avanzar al siguiente. Este enfoque permite detectar problemas de forma temprana y limitar su impacto sobre el conjunto del sistema.

Para mitigar los riesgos asociados al aprendizaje autónomo, se ha priorizado la documentación y el estudio previo de las tecnologías clave antes de su integración efectiva en el proyecto. Asimismo, se ha planteado una planificación flexible que permite ajustar el orden de los requisitos en función de la complejidad real encontrada durante el desarrollo.

En relación con los riesgos derivados del procesamiento de documentos y la calidad de los resultados, se han definido pruebas funcionales básicas que permiten validar el comportamiento esperado del sistema y detectar posibles limitaciones sin necesidad de implementar soluciones excesivamente complejas.

Desde el punto de vista organizativo, la principal medida de mitigación consiste en una planificación realista del proyecto, con una estimación temporal ajustada al nivel de dedicación disponible. El uso de Kanban como metodología de seguimiento permite además visualizar el estado del proyecto en todo momento y detectar desviaciones de forma temprana.

Por último, para gestionar las posibles desviaciones entre planificación y ejecución, se ha previsto un capítulo específico de seguimiento y análisis de desviaciones, donde se documentan las diferencias entre lo previsto y lo realizado, así como las decisiones adoptadas para garantizar la finalización del proyecto.

Con el objetivo de estructurar el análisis de riesgos de forma clara y evaluable, se presenta a continuación una tabla resumen que recoge los principales riesgos

identificados, su probabilidad, su impacto estimado y las medidas de mitigación previstas.

Tabla 4.1. Resumen de riesgos identificados

Riesgo	Probabilidad	Impacto	Nivel	Medidas de Mitigación
Dificultades en la integración de múltiples componentes (backend, vectorización, IA)	Media	Alto	Medio-Alto	Desarrollo incremental por bloques y validación temprana de cada módulo
Curva de aprendizaje en tecnologías no abordadas en el ciclo	Alta	Medio	Medio-Alto	Estudio previo, prototipos pequeños antes de integración definitiva
Resultados no óptimos en búsqueda semántica o generación de respuestas	Media	Medio	Medio	Pruebas funcionales básicas y ajustes progresivos de configuración
Desviación temporal respecto a la planificación inicial	Media	Medio	Medio	Seguimiento continuo mediante Kanban y revisión periódica de prioridades
Limitación de recursos al tratarse de un desarrollo individual	Alta	Medio	Medio-Alto	Planificación realista y priorización de funcionalidades esenciales
Problemas en el procesamiento de determinados formatos de documento	Baja-Media	Medio	Bajo-Medio	Restricción inicial a formatos soportados y validación controlada

Los riesgos identificados se consideran coherentes con el alcance técnico y organizativo del proyecto. Las medidas de mitigación definidas no eliminan completamente la incertidumbre inherente al desarrollo software, pero permiten reducir su impacto y garantizar una gestión responsable del proyecto.

CIERRE DEL CAPÍTULO

El análisis de riesgos presentado no pretende eliminar la incertidumbre inherente al desarrollo de un proyecto software, sino demostrar una aproximación madura y consciente a su gestión. La identificación temprana de riesgos y la definición de medidas de mitigación coherentes permiten reducir el impacto de los problemas potenciales y refuerzan la planificación global del proyecto.

ARQUITECTURA Y DISEÑO DEL SISTEMA (R03)

El presente capítulo describe la arquitectura global del sistema desarrollado, identificando sus principales componentes, las relaciones entre ellos y las decisiones técnicas adoptadas en su diseño. El objetivo no es detallar la implementación interna de cada módulo, sino ofrecer una visión estructurada y comprensible del sistema desde un punto de vista conceptual y técnico.

La arquitectura se ha definido antes del inicio del desarrollo, con el fin de garantizar coherencia entre los requisitos funcionales definidos en fases anteriores y la solución técnica propuesta. Este enfoque permite reducir improvisaciones, facilitar la integración de componentes y asegurar la trazabilidad entre diseño y desarrollo.

VISIÓN GENERAL DEL SISTEMA

El sistema desarrollado puede entenderse como una plataforma de gestión documental enriquecida con capacidades de búsqueda semántica y generación de respuestas contextualizadas. Desde una perspectiva de alto nivel, el sistema se compone de los siguientes bloques principales:

- Interfaz de usuario (frontend básico)
- API backend
- Sistema de almacenamiento documental
- Base de datos relacional con soporte vectorial
- Módulo de procesamiento de documentos
- Módulo de búsqueda semántica
- Flujo de generación de respuestas (RAG)

El usuario interactúa con el sistema a través de una interfaz que permita realizar operaciones como la subida de documentos, la consulta de la información y la ejecución de búsquedas. Estas peticiones son gestionadas por el backend, que actúa como núcleo de coordinación del sistema.

El backend se encarga de:

- Gestionar usuarios y autenticación
- Controlar roles y permisos
- Orquestar el procesamiento documental

- Gestionar consultas semánticas
- Integrar el flujo de generación de respuestas

La base de datos cumple una doble función: almacenar información estructurada (usuarios, metadata, versiones) y persistir representaciones vectoriales de los fragmentos de texto procesados.

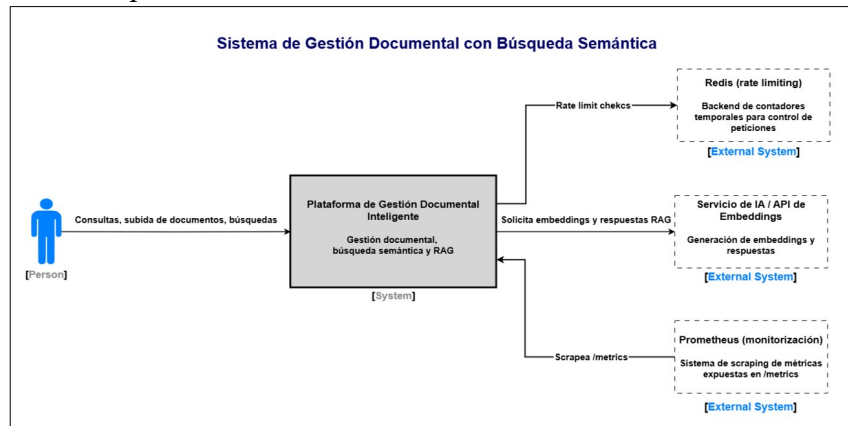


Figura 5.1 – Diagrama C4 Nivel Contexto.

Representación del sistema como una única entidad dentro de su entorno, mostrando la interacción con el usuario y el servicio externo de IA empleado para la generación de embeddings y respuestas.

A continuación, se presenta el diagrama C4 de nivel contenedor, donde se descompone el sistema en sus principales elementos desplegables y se detallan las relaciones entre ellos.

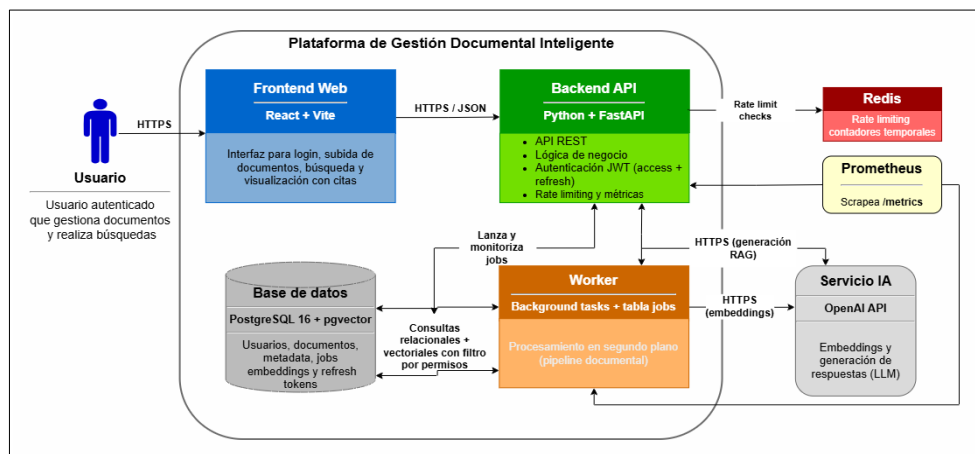


Figura 5.2 – Diagrama C4 Nivel Contenedor.

Descomposición del sistema en sus principales contenedores: frontend, backend, base de datos, procesamiento en segundo plano, e infraestructura auxiliar compuesta por Redis (rate limiting), Prometheus (monitoreación mediante métricas) y servicio externo de IA).

El diagrama anterior muestra la separación del sistema en contenedores desplegados claramente diferenciados. El frontend web actúa como cliente del backend, gestionando la interacción con el usuario mediante peticiones HTTPS en formato JSON. Toda la lógica de negocio y el control de acceso se centralizan en el backend API, evitando el acceso directo a la base de datos y garantizando una arquitectura controlada. Las métricas del backend se exponen mediante el endpoint `/metrics`, permitiendo su recolección automática por Prometheus.

La persistencia se concentra en PostgreSQL con soporte vectorial, donde conviven los datos relacionales (usuarios, documentos, metadata y estados de procesamiento) y los embeddings asociados a los fragmentos de texto. Esta unificación permite implementar búsquedas semánticas e híbridas sin necesidad de integrar motores externos adicionales.

El procesamiento documental se ejecuta en segundo plano mediante un componente de tipo *worker*, encargado de la extracción de texto, fragmentación y vectorización. Este diseño evita bloquear las operaciones síncronas del backend y permite mantener trazabilidad del estado de cada trabajo.

Finalmente, el servicio externo de IA se utiliza exclusivamente para la generación de embeddings y respuestas, manteniéndose fuera del dominio de la aplicación y desacoplado del resto de la infraestructura.

ARQUITECTURA LÓGICA

La arquitectura lógica del sistema se ha diseñado siguiendo un modelo de capas, con el objetivo de separar responsabilidades y facilitar el mantenimiento del código. Esta separación permite aislar cambios, mejorar la claridad estructural y reducir acoplamientos innecesarios.

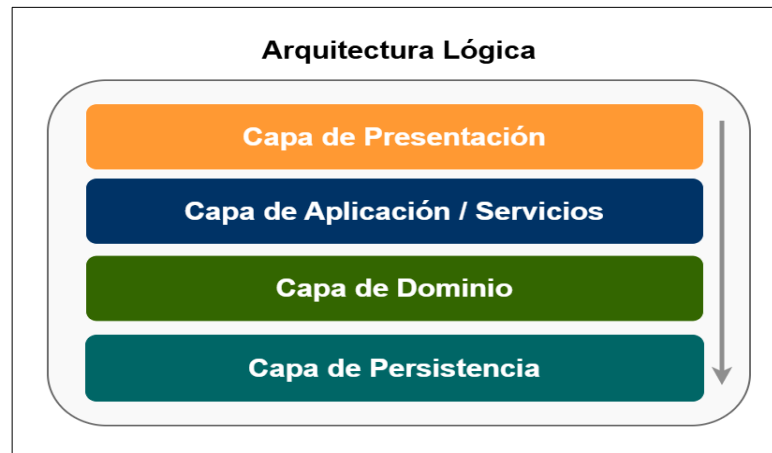


Figura 5.3 – Diagrama de Arquitectura Lógica en Capas.
Organización interna del backend en separación de responsabilidades

Capa de presentación

Responsable de la interacción con el usuario. Incluye la interfaz básica de búsqueda y gestión documental. Su función es recoger las solicitudes del usuario y mostrar resultados, sin incorporar lógica de negocio compleja.

Capa de aplicación o servicios

Contiene la lógica principal del sistema. Aquí se gestionan los casos de uso, la coordinación entre módulos y la validación de reglas de negocio. Esta capa actúa como intermediaria entre la presentación y la persistencia.

Capa de dominio

Define las entidades principales del sistema y sus relaciones conceptuales. Incluye modelos como Usuario, Documento, Versión, Metadata, Chunk y Embedding.

Capa de persistencia

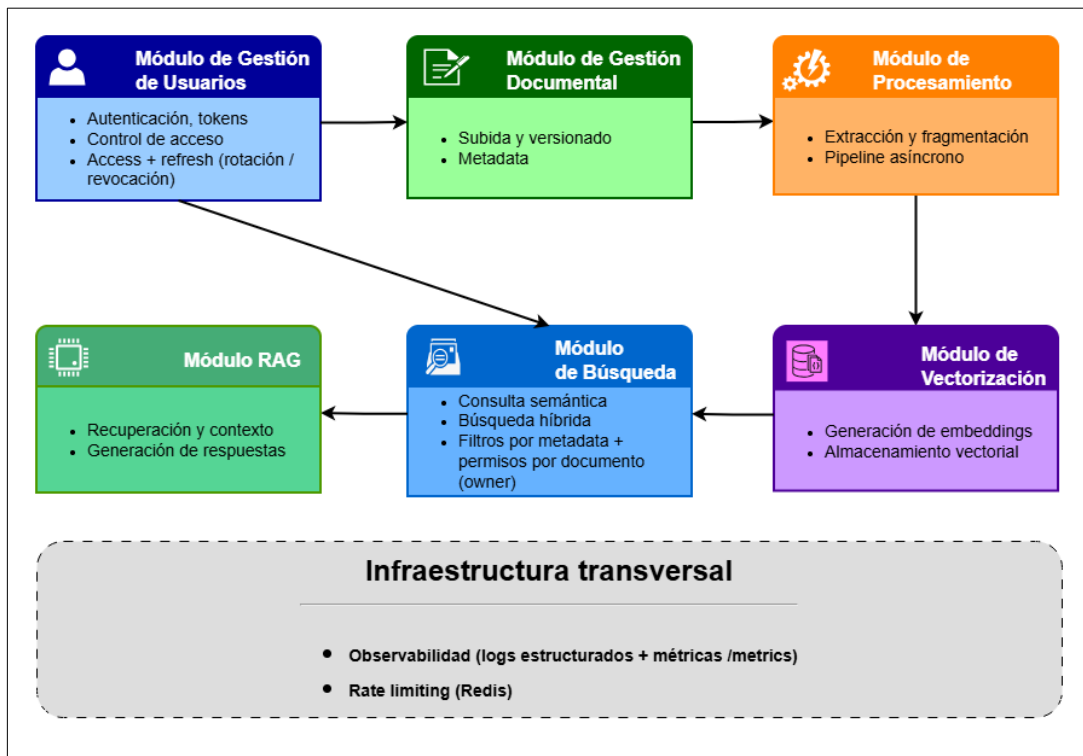
Gestiona el acceso a la base de datos, incluyendo tanto el almacenamiento relacional tradicional como el almacenamiento vectorial.

Esta arquitectura en capas permite cumplir con el principio de separación de responsabilidades, facilitando la evolución futura del sistema y evitando dependencias circulares.

COMPONENTES PRINCIPALES

La Figura 5.4 muestra donde se representan los módulos funcionales identificados y sus relaciones principales. El diagrama permite visualizar el flujo de información y la separación de responsabilidades dentro de la plataforma.

A partir de la arquitectura lógica se identifican los siguientes componentes funcionales clave del sistema:



En la Figura 5.4 – Diagrama de Componentes Principales del Sistema

Como se observa en la figura, el sistema se organiza en módulos especializados que colaboran entre sí para cubrir el ciclo completo de gestión documental inteligente. A continuación, se describe en detalle la responsabilidad y el funcionamiento de cada uno de los componentes representados.

MÓDULO DE GESTIÓN DE USUARIOS

Encargado de la autenticación, la generación y validación de *tokens* (access + refresh), el soporte de rotación y revocación de refresh tokens, y el control de acceso basado en roles. Este componente proporciona una capa transversal de seguridad, garantizando que únicamente los usuarios autorizados puedan acceder a los recursos y *endpoints* sensibles del sistema.

MÓDULO DE GESTIÓN DOCUMENTAL

Responsable de la subida de documentos, el versionado, la asociación de metadatos y la persistencia de la información estructurada. Constituye la base funcional del sistema, ya que sobre él se apoyan las capacidades de procesamiento y explotación semántica posteriores.

MÓDULO DE PROCESAMIENTO

Incluye la extracción de texto desde los documentos, su fragmentación en unidades menores (*chunks*) y la orquestación asíncrona del pipeline de tratamiento. Su finalidad es transformar documentos en información estructurada y preparada para su posterior análisis y vectorización.

MÓDULO DE VECTORIZACIÓN

Genera embeddings a partir de los fragmentos de texto y los almacena en la base de datos con soporte vectorial. Este componente permite representar el contenido en un espacio semántico, haciendo posible la recuperación basada en similitud.

MÓDULO DE BÚSQUEDA

Proporciona mecanismos de consulta semántica y búsqueda híbrida (texto + vector), incorporando filtrado por metadatos y restricciones de acceso por documento. Actúa como punto central de recuperación de información estructurada y vectorial, garantizando que los fragmentos devueltos respetan los permisos de usuario autenticado. En esta versión, el control de acceso se aplica por propiedad (owner), de forma que únicamente se recuperan fragmentos pertenecientes a documentos autorizados para el usuario. Modelos más avanzados como ACL o multi-tenant (tenant) se contemplan como posibles ampliaciones futuras.

MÓDULO RAG

Coordina la recuperación de fragmentos relevantes, la construcción de contexto y la generación de respuestas mediante un modelo de lenguaje. Este módulo se apoya en los anteriores sin sustituirlos, reforzando el diseño modular y la separación de responsabilidades del sistema.

DISEÑO DE DATOS

El diseño de datos se ha planteado siguiendo un modelo relacional extendido con capacidades vectoriales. Las entidades principales incluyen:

- Usuario
- Rol
- Documento
- Versión
- Metadata
- Chunk
- Embedding
- Evento (tracking)
- Refresh_Token

La Figura 5.5 muestra el diagrama simplificado del modelo de datos propuesto, donde se representan las entidades principales del sistema y las relaciones existentes entre ellas.

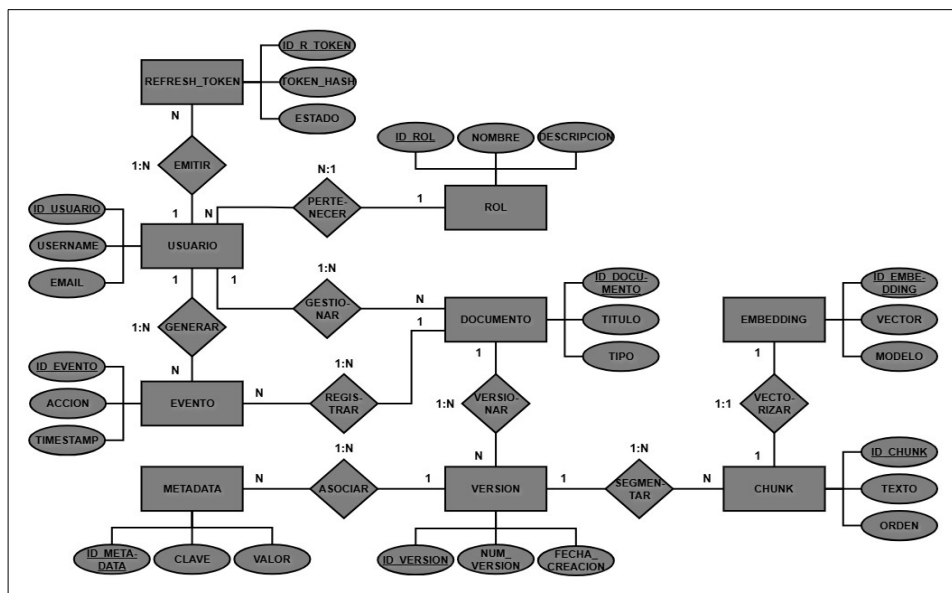


Figura 5.5 – Diagrama de Modelo de Datos E/R

El diagrama E/R muestra las relaciones principales del modelo: documentos, versiones y chunks, junto con la asociación uno a uno entre cada fragmento y su embedding vectorial para garantizar trazabilidad. También se incluye la entidad Refresh_Token vinculada a Usuario para la gestión segura de sesiones. La separación entre datos relacionales y representación vectorial permite extender el modelo con capacidades semánticas manteniendo la coherencia del diseño.

DECISIONES TÉCNICAS ADOPTADAS

La selección tecnológica del proyecto se ha realizado atendiendo a criterios de coherencia arquitectónica, viabilidad de implementación en un contexto académico y alineación con tecnologías ampliamente utilizadas en entornos profesionales actuales. No se ha buscado incorporar herramientas por tendencia o novedad, sino aquellas que permiten cumplir los requisitos definidos de manera estructurada, mantenible y justificable.

La arquitectura propuesta combina un backend moderno orientado a servicios, una base de datos relacional con capacidades vectoriales, un pipeline de procesamiento documental modular y un entorno de despliegue reproducible. Las decisiones adoptadas buscan equilibrio entre robustez técnica y complejidad razonable para un desarrollo individual.

A continuación, se detallan las principales decisiones técnicas adoptadas, justificando su elección y el descarte de las alternativas consideradas.

Tabla 5.1 – Tecnologías seleccionadas y alternativas descartadas

Área	Tecnología elegida	Por qué esta	Alternativa descartada
Backend API	Python + FastAPI	Ecosistema IA y rapidez de desarrollo	Java + Spring Boot
Servidor ASGI	Uvicorn + Gunicorn	Entorno productivo y compatible con contenedores	Servidor embebido simple
Base de datos	PostgreSQL 16	Robustez relacional y extensibilidad	MongoDB
Búsqueda vectorial	pgvector	Unificación relacional + vectorial	Motor vectorial externo
ORM	SQLAlchemy 2.0 + Alembic	Control de modelo y migraciones versionadas	Acceso directo sin ORM
Procesamiento documental	PyMuPDF + python-docx	Extracción fiable y ligera	OCR completo
Autenticación	JWT (access + refresh) + RBAC	Seguridad stateless con renovación segura	Sesiones en servidor
Pipeline asíncrono	Background tasks + tabla jobs	Simplicidad sin infraestructura adicional	Celery + Redis
Rate limiting	Redis	Contadores rápidos y atómicos sin cargar la BD	Rate limiting en PostgreSQL
Observabilidad	Prometheus	Recolección de métricas de la API vía /metrics	Monitorización cloud externa
Embeddings /	OpenAI API	Integración estable y rápida	Modelo local autoalojado

Área	Tecnología elegida	Por qué esta	Alternativa descartada
IA			
Frontend	React + Vite	Ligero y suficiente para validación funcional	Framework SSR complejo
Contenerización	Docker Compose	Reproducibilidad y coherencia DevOps	Instalación manual
Testing	pytest	Ecosistema consolidado en Python	Tests ad-hoc
Control de versiones	Git + GitHub	Trazabilidad y control de desarrollo	Control local sin remoto

***Nota:** Redis se incorpora únicamente como backend para el mecanismo de rate limiting y no como cola de tareas del pipeline asíncrono.*

BACKEND Y ARQUITECTURA DE SERVICIOS

El backend se ha implementado utilizando Python y el framework FastAPI, orientado a la construcción de APIs REST modernas. Esta elección permite estructurar el sistema de endpoints claramente definidos, favoreciendo la separación de responsabilidades y la modularidad del código.

FastAPI ofrece validación automática de datos mediante modelos tipados y generación automática de documentación OpenAPI, lo que facilita tanto el desarrollo como la validación funcional del sistema. Su integración natural en el ecosistema Python resulta especialmente adecuada para proyectos que incorporan procesamiento de texto y técnicas de inteligencia artificial.

La alternativa Java con Spring Boot se consideró desde un punto de vista empresarial; sin embargo, se descartó por introducir una mayor complejidad estructural sin aportar ventajas significativas para el alcance académico del proyecto.

PERSISTENCIA Y MODELO DE DATOS

La base de datos seleccionada es PostgreSQL, complementada con la extensión pgvector para el almacenamiento de embeddings. Esta decisión permite mantener tanto los datos estructurados (usuarios, documentos, versiones, metadata) como las representaciones vectoriales en un único sistema de persistencia.

Esta unificación simplifica la arquitectura y facilita la trazabilidad entre documentos y resultados de búsqueda semántica. Además, PostgreSQL proporciona robustez

transaccional y un modelo relacional coherente con el diseño de datos definido en el apartado anterior.

Se descartó el uso de motores externos de búsqueda o bases de datos vectoriales independientes, ya que introducirían complejidad adicional y fragmentarían la arquitectura sin aportar un beneficio proporcional en el contexto del proyecto.

SEGURIDAD Y CONTROL DE ACCESO

La autenticación se basa en tokens JWT con un modelo de control de acceso basado en roles (RBAC), manteniendo una arquitectura stateless y protegiendo los endpoints del sistema de forma granular.

Para reforzar la seguridad y permitir sesiones persistentes sin recurrir a estado en servidor, se incorpora un esquema de access token de vida corta (10-20 minutos) y refresh token de vida más larga (7-30 días). El refresh token permite obtener nuevos access tokens sin requerir un login completo.

Con el fin de mitigar riesgos de reutilización indebida, se aplica rotación de refresh tokens: cada refresh válido utilizado se invalida y se emite uno nuevo. Además, se contempla revocación (logout o sospecha), almacenando en base de datos el refresh token de forma segura mediante hash, junto con información de estado (revocado, reemplazado o expirado).

Si bien el RBAC controla el acceso a funcionalidades y endpoints, el sistema aplica también el principio de mínimo privilegio en el acceso a la información documental: en el contexto de búsqueda semántica y RAG, la recuperación de chunks se filtra en función de los permisos sobre documentos, garantizando que un usuario solo pueda recuperar información de documentos autorizados.

Como medida adicional de seguridad, se implementa rate limiting sobre endpoints críticos (autenticación, subida documental y consultas intensivas), utilizando Redis como backend de almacenamiento temporal para contadores con expiración.

PROCESAMIENTO DOCUMENTAL Y PIPELINE

El procesamiento de documentos se implementa mediante librerías especializadas en extracción de texto, integradas dentro de un pipeline modular. Se ha optado por un

modelo de tareas en segundo plano gestionado mediante funciones asíncronas y una tabla de seguimiento de estados.

Esta solución permite mantener un flujo controlado sin introducir dependencias adicionales como sistemas de mensajería externos. Aunque herramientas como Celery y Redis podrían aportar mayor escalabilidad, se consideró que excedían el alcance y complejidad razonable del proyecto.

VECTORIZACIÓN E INTEGRACIÓN DE IA

Para la generación de embeddings y el flujo RAG se ha optado por integrar un modelo accesible mediante API externa. Esta decisión permite centrarse en la arquitectura de recuperación y control de contexto sin asumir la gestión de infraestructura de modelos locales.

Se ha priorizado la estabilidad, documentación y facilidad de integración frente a soluciones experimentales o autoalojadas que aumentarían significativamente la carga técnica.

ENTORNO DE EJECUCIÓN Y CALIDAD

El sistema se despliega mediante Docker Compose, lo que garantiza reproducibilidad y coherencia entre entornos. El uso de contenedores facilita la evaluación del proyecto y refuerza la alineación con prácticas profesionales actuales.

Se han incorporado pruebas automatizadas mediante pytest y control de versiones mediante Git y repositorio remoto, asegurando trazabilidad del desarrollo. Además, se incorporan mecanismos de observabilidad para facilitar el análisis del comportamiento del sistema. Se generan logs estructurados en formato JSON incluyendo *request_id*, *user_id* (si aplica), endpoint, código de respuesta y latencia, así como información por etapa del pipeline (retrieval, embedding, LLM). Asimismo, se expone un endpoint */metrics* compatible con Prometheus para el registro de métricas de peticiones HTTP y del flujo RAG.

Para proteger recursos y evitar abuso del sistema, se implementa rate limiting sobre endpoints críticos como */auth/login*, */query* y */documents/upload*, devolviendo HTTP 429 cuando supera el umbral configurado.

CIERRE DEL APARTADO

Las decisiones técnicas adoptadas reflejan un enfoque equilibrado entre solidez arquitectónica, viabilidad académica y alineación con tecnologías demandadas en entornos profesionales actuales. Cada elección responde a una necesidad concreta del sistema y se integra de forma coherente en la arquitectura global definida en este capítulo.

Asimismo, todas las tecnologías seleccionadas permiten una evolución futura del sistema sin necesidad de rediseñar la arquitectura base.

DESARROLLO DEL SISTEMA (R10 – R56)

El desarrollo del sistema constituye el núcleo técnico del proyecto y recoge la implementación progresiva de los distintos bloques funcionales definidos durante la fase de planificación. El proyecto se ha estructurado siguiendo un enfoque incremental, implementando primero las funcionalidades base del backend y extendiendo posteriormente el sistema hacia capacidades de búsqueda semántica y recuperación inteligente de información.

Este enfoque permite mantener la coherencia con el modelo RFTP definido previamente, estableciendo una relación directa entre requisitos, funcionalidades implementadas y evidencias de validación. Cada bloque funcional se ha desarrollado como una unidad relativamente independiente, facilitando el control de complejidad y la trazabilidad técnica del proyecto.

El desarrollo se divide en dos grandes áreas. Por una parte, el backend core, responsable de la seguridad, gestión documental, procesamiento e indexación de la información. Por otra, el sistema de búsqueda, orientado a la recuperación eficiente de información mediante técnicas semánticas e híbridas.

La implementación se ha realizado priorizando la estabilidad del sistema y la separación de responsabilidades entre módulos, evitando acoplamientos innecesarios y permitiendo la evolución progresiva del proyecto. Este diseño modular ha facilitado la integración posterior de componentes avanzados basados en inteligencia artificial.

BACKEND CORE

El backend constituye el núcleo operativo del sistema y concentra la mayor parte de la lógica de negocio. Su desarrollo se ha realizado siguiendo una estructura modular organizada por responsabilidades funcionales, permitiendo separar claramente las áreas de seguridad, gestión documental, procesamiento de datos y almacenamiento vectorial. El objetivo principal de este bloque es proporcionar una base sólida sobre la que construir las funcionalidades avanzadas del sistema, garantizando desde las primeras fases aspectos esenciales como la autenticación, el control de acceso, la persistencia de datos y la trazabilidad del procesamiento documental.

Dentro del backend core se distinguen cuatro áreas principales:

- Seguridad y gestión de usuarios.
- Gestión documental.
- Ingesta y procesamiento de documentos.
- Vectorización y almacenamiento vectorial.

Cada una de estas áreas se desarrolla de forma progresiva, manteniendo dependencias claras entre requisitos y asegurando que cada módulo pueda validarse antes de avanzar al siguiente.

SEGURIDAD Y GESTIÓN DE USUARIOS (R10 – R15)

Modelo de usuarios (R10)

El primer elemento implementado dentro del bloque de seguridad es el modelo de usuarios del sistema, que constituye la base para los mecanismos de autenticación y autorización desarrollados posteriormente.

Se definió una entidad *User* persistente utilizando SQLAlchemy 2.0 como ORM, permitiendo mapear objetos Python a tablas relacionales en PostgreSQL. La entidad incluye los siguientes atributos principales:

- Identificador único (UUID) como clave primaria.
- Dirección de correo electrónico única.
- Hash de contraseña.
- Indicador de activación de usuario.

- Timestamps de creación y actualización.

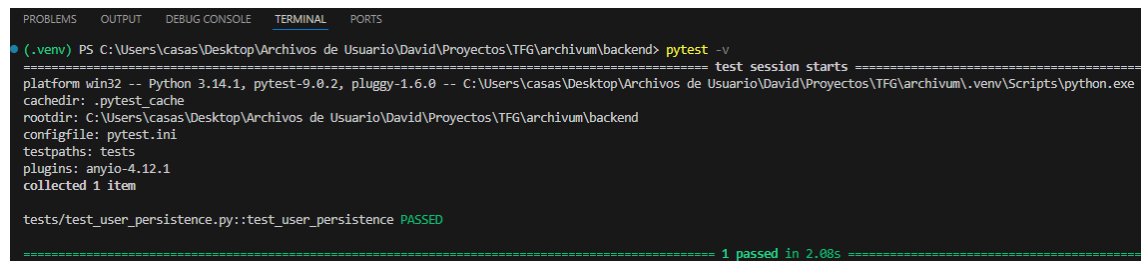
Se optó por utilizar UUID como identificador primario en lugar de un entero autoincremental, con el objetivo de mejorar la unicidad global y evitar exposición de secuencias predecibles.

La persistencia del modelo se gestiona mediante un repositorio de acceso a datos, desacoplando la lógica de negocio de la capa de almacenamiento. Para la gestión de sesiones y conexión a base de datos se configura un motor mediante SQLAlchemy, garantizando control explícito de transacciones.

Asimismo, la estructura de la base de datos se versionó mediante Alembic, permitiendo generar y aplicar migraciones de forma controlada. Esto asegura trazabilidad en la evolución del modelo y coherencia entre entornos.

La funcionalidad fue validada mediante pruebas automatizadas que verifican la creación y recuperación de usuarios en base de datos, confirmando la correcta persistencia de los datos.

La validación del modelo se realizó mediante una prueba automatizada con pytest, verificando la creación y recuperación de un usuario en base de datos.



```

(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.12.1
collected 1 item

tests/test_user_persistence.py::test_user_persistence PASSED

===== 1 passed in 2.08s =====
  
```

Figura 6.1 – Ejecución satisfactoria del test de persistencia del modelo de usuario

Este modelo sirve como base estructural para los requisitos R11 y R12.

Autenticación JWT + Refresh Tokens (R11)

Una vez definido el modelo de usuario en el requisito anterior, se implementa el sistema de autenticación del backend, encargado de verificar la identidad de los usuarios y controlar el acceso a los distintos endpoints de la aplicación.

Para ello, se adopta un enfoque basado en JSON Web Tokens (JWT), que permite mantener un sistema de autenticación sin estado (stateless), evitando la necesidad de

gestionar sesiones en servidor y facilitando la escalabilidad del sistema, siguiendo las buenas prácticas de seguridad en APIs REST.

El proceso de autenticación comienza con el endpoint de login, donde el usuario proporciona sus credenciales (correo electrónico y contraseña). Estas credenciales se validan contra la información almacenada en base de datos, comparando la contraseña proporcionada con el hash persistido mediante el uso de la librería *passlib*. Si la verificación es correcta, el sistema genera dos tokens: un *access token* y un *refresh token*.

El *access token* tiene una duración limitada y contiene la información mínima necesaria para identificar al usuario, incluyendo su identificador y el tipo de token. Este token se firma utilizando un algoritmo criptográfico (HS256) y se envía al cliente, que lo utilizará en las siguientes peticiones mediante la cabecera **Authorization** con el esquema Bearer.

Para proteger los endpoints del sistema, se implementa un mecanismo de validación que intercepta las peticiones entrantes. Este proceso extrae el token de la cabecera, verifica su firma, comprueba su expiración y valida su tipo. Si el token es válido, se permite el acceso al recurso solicitado; en caso contrario, se devuelve un error de autenticación. Este enfoque garantiza que únicamente los usuarios autenticados puedan acceder a los endpoints protegidos.

Con el objetivo de evitar que el usuario tenga que iniciar sesión continuamente, se introduce el uso de *refresh tokens*. A diferencia del access token, el refresh token tiene una duración mayor y se utiliza exclusivamente para obtener nuevos tokens sin necesidad de volver a introducir credenciales.

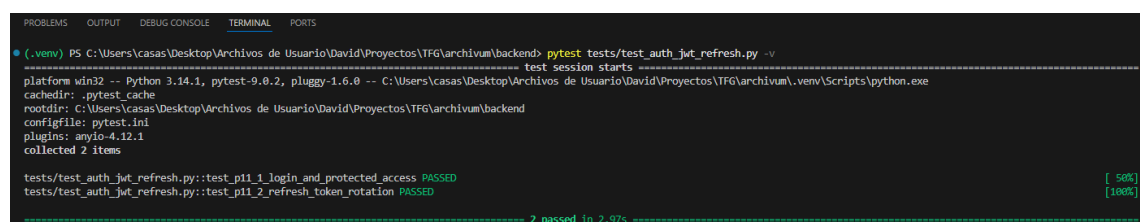
El refresh token incluye un identificador único (jti) y se almacena en base de datos de forma segura mediante un hash, junto con su estado (activo, revocado o reemplazado). Esto permite mantener el control sobre los tokens emitidos y ampliar medidas de seguridad adicionales.

Cuando el cliente necesita renovar su sesión, envía el refresh token al endpoint correspondiente. El sistema valida el token y, si es correcto, genera un nuevo access token y un nuevo refresh token. En este proceso se aplica una estrategia de rotación: el

refresh token utilizado se invalida y se reemplaza por uno nuevo. De esta forma, se reduce el riesgo de reutilización indebida de tokens en caso de filtración.

Además, si un refresh token ya revocado es reutilizado, el sistema lo detecta y rechaza la petición, evitando posibles ataques de tipo replay. Este comportamiento refuerza la seguridad del sistema sin necesidad de mantener sesiones activas en servidor.

La funcionalidad implementada se ha validado mediante pruebas automatizadas utilizando pytest, donde se verifica el flujo completo de autenticación: creación de usuario, login, acceso a endpoints protegidos, renovación de tokens y comprobación de invalidación del refresh token anterior tras la rotación.



```
PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest tests/test_auth_jwt_refresh.py -v
test session starts
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 2 items

tests/test_auth_jwt_refresh.py::test_pi1_1_login_and_protected_access PASSED [ 50%]
tests/test_auth_jwt_refresh.py::test_pi1_2_refresh_token_rotation PASSED [100%]

----- 2 passed in 2.97s -----
```

Figura 6.2 – Ejecución satisfactoria de los test de autenticación (login, acceso protegido y refresh token)

El sistema resultante proporciona un mecanismo de autenticación robusto y alineado con prácticas actuales de desarrollo backend, permitiendo gestionar sesiones de forma segura y eficiente. Este bloque sirve como base para los requisitos posteriores relacionados con el control de acceso y seguridad del sistema.

Autorización basada en roles y permisos por documento (R12)

En esta fase se ha implementado un sistema de autorización que combina control de acceso basado en roles (RBAC) con un modelo de permisos aplicado a nivel de documento (ACL simple). Este requisito complementa directamente la autenticación definida en R11, permitiendo no solo identificar al usuario, sino también determinar qué acciones puede realizar y sobre qué recursos.

En primer lugar, se ha definido un modelo de roles del sistema, estableciendo tres perfiles principales: **admin**, **editor** y **viewer**. Estos roles representan distintos niveles de acceso funcional dentro de la aplicación. La relación entre usuarios y roles se ha implementado mediante una asociación de tipo muchos a muchos, permitiendo que un mismo usuario pueda disponer de varios roles simultáneamente si fuese necesario.

A partir de esta estructura, se ha aplicado un control de acceso basado en roles (RBAC) sobre los endpoints del backend. Para ello, se ha desarrollado una capa de validación reutilizable que comprueba si el usuario autenticado posee alguno de los roles requeridos antes de permitir la ejecución de determinadas operaciones. De este modo, funcionalidades sensibles como la gestión de usuarios o la creación de documentos quedan restringidas a perfiles concretos, garantizando una separación clara de responsabilidades.

Sin embargo, el control de acceso únicamente basado en roles resulta insuficiente en un sistema orientado a gestión documental, donde el acceso a los recursos depende también del contexto. Por este motivo, se ha incorporado un modelo de permisos por documento (ACL simple), que introduce una capa adicional de seguridad a nivel de recurso.

Cada documento del sistema se asocia a un usuario propietario (**owner**), que dispone de control total sobre el mismo. Además, se ha definido una tabla de accesos explícitos que permite establecer reglas de acceso sencillas pero efectivas:

- El usuario con rol admin puede acceder a todos los documentos del sistema.
- El propietario de un documento puede acceder y gestionarlo en todo momento.
- Un usuario puede acceder a un documento si ha recibido un permiso explícito sobre el mismo.
- En caso contrario, el acceso es denegado.

La lógica de autorización se ha centralizado en la capa de servicio, evitando duplicidad de validaciones en los endpoints y facilitando el mantenimiento del sistema. Los endpoints del backend utilizan esta lógica para filtrar automáticamente los recursos visibles y bloquear accesos no autorizados, devolviendo códigos de error adecuados cuando procede.

Adicionalmente, se ha implementado un filtrado de resultados en las operaciones de consulta, de forma que cada usuario únicamente puede visualizar aquellos documentos sobre los que tiene permisos. Esto resulta especialmente relevante de cara a futuras funcionalidades como la búsqueda semántica, donde será necesario garantizar que los resultados devueltos respeten las restricciones de acceso definidas.

Finalmente, el correcto funcionamiento del sistema se ha validado mediante pruebas automatizadas que cubren tanto el control de acceso por rol como el acceso a documentos en función de la propiedad y los permisos explícitos. Estas pruebas verifican escenarios de acceso autorizado y no autorizado, asegurando que las restricciones se aplican de forma consistente.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r12_rbac_acl.py
test session starts
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 2 items

tests/test_r12_rbac_acl.py::test_pi2_1_rbac_authorized_and_unauthorized_access PASSED [ 50%]
tests/test_r12_rbac_acl.py::test_pi2_2_document_access_by_ownership_and_acl PASSED [100%]

===== 2 passed in 4.59s =====
```

Figura 6.3 – Verificación del sistema de autorización basado en RBAC y permisos por documento (ACL), con resultado satisfactorio.

En conjunto, la combinación de RBAC con un modelo ACL simple permite implementar un sistema de autorización suficientemente robusto para el alcance del proyecto, manteniendo una complejidad controlada y alineada con los objetivos del Proyecto. Este enfoque sienta las bases para la aplicación de seguridad contextual en fases posteriores del sistema.

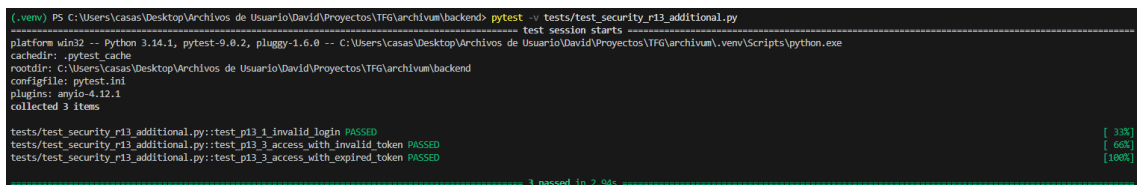
Tests de seguridad (R13)

Una vez implementados el modelo de usuarios, la autenticación basada en JWT con refresh tokens y el sistema de autorización con roles y permisos documentales, se llevó a cabo una fase específica de validación orientada a comprobar el comportamiento conjunto del bloque de seguridad del sistema. El objetivo de este requisito no fue duplicar las pruebas ya realizadas en R10, R11 y R12, sino completar su validación mediante una batería adicional centrada en escenarios negativos relevantes.

La validación de R13 se apoyó, por un lado, en las pruebas ya existentes del proyecto para persistencia de usuarios, autenticación con acceso protegido, rotación de refresh tokens y control de acceso por roles y ownership. Sobre esta base, se añadió un archivo de pruebas complementario orientado específicamente a tres casos de seguridad no cubiertos de forma explícita en los requisitos anteriores: intento de login con credenciales inválidas, acceso a endpoint protegido con token manipulado o inexistente, y acceso con token expirado.

En el primer caso, se verificó que el endpoint de autenticación rechaza correctamente credenciales erróneas, devolviendo un código HTTP 401 y evitando la emisión de tokens. En el segundo, se comprobó que un token no válido no permite acceder a recursos protegidos. Finalmente, se generó de forma controlada un token de acceso expirado para confirmar que el backend también rechaza tokens correctamente firmados pero fuera de su ventana temporal de validez.

Este enfoque permitió reforzar la validación del sistema desde una perspectiva de seguridad defensiva, comprobando no solo el funcionamiento esperado en escenarios correctos, sino también la respuesta del backend ante intentos de acceso no autorizados. De este modo, R13 quedó alineado con su función dentro del proyecto: validar de forma conjunta y coherente el bloque R10-R12 antes de continuar con requisitos posteriores relacionados con gestión documental, procesamiento y búsqueda. La memoria sitúa R13 dentro del bloque de seguridad y usuarios de la fase 1, con una estimación inicial de 4 horas, y lo define expresamente como validación del funcionamiento conjunto de autenticación y autorización.



```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_security_r13_additional.py
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 3 items

tests/test_security_r13_additional.py::test_p13_1_invalid_login PASSED [ 33%]
tests/test_security_r13_additional.py::test_p13_3_access_with_invalid_token PASSED [ 66%]
tests/test_security_r13_additional.py::test_p13_3_access_with_expired_token PASSED [100%]

===== 3 passed in 2.94s =====
```

Figura 6.4 – Verificación complementaria de los tests de seguridad del sistema, centrada en escenarios negativos de autenticación y validación de tokens, con resultado satisfactorio.

Rate limiting en endpoints críticos (R14)

Tras la implementación de la autenticación mediante JWT, la gestión de refresh tokens y el control de acceso basado en roles, se incorporó un mecanismo adicional orientado a proteger la API frente a usos abusivos. Este requisito (R14) introduce un sistema de limitación de peticiones sobre endpoints críticos con el objetivo de evitar ataques de fuerza bruta, consumo excesivo de recursos y degradación del servicio.

Aunque los mecanismo de autenticación y autorización permiten controlar quién accede al sistema y a qué recursos, no regulan la frecuencia de uso. Por ello, se diseñó una solución basada en un middleware transversal que intercepta las peticiones antes de que alcancen la lógica de negocio y verifica si se ha superado un umbral de solicitudes dentro de una ventana temporal determinada.

El rate limiting se aplicó sobre tres endpoints principales. En primer lugar, */auth/login*, donde la limitación se realiza por dirección de IP, ya que no existe todavía un usuario autenticado. Esta medida permite mitigar intentos repetidos de acceso no autorizado. En segundo lugar, */query*, donde la limitación se basa en el usuario autenticado, utilizando la información contenida en el token, con fallback a la IP en caso de ausencia de autenticación. Por último, */documents/upload*, donde también se aplica una limitación por usuario debido al mayor coste asociado a la subida de archivos.

Para gestionar los contadores de peticiones se optó por Redis como sistema de almacenamiento en memoria. Esta elección permite realizar operaciones de incremento atómico y establecer tiempos de expiración de forma eficiente, evitando sobrecargar la base de datos principal con información temporal. Cada petición genera una clave identificativa que combina endpoint, método HTTP e identificador del cliente, sobre la que se incrementa un contador asociado a una ventana temporal configurable.

Cuando el número de peticiones supera el límite establecido, el middleware interrumpe el flujo normal de ejecución y devuelve una respuesta HTTP 429 (*“Too Many Request”*), acompañada de un mensaje descriptivo y cabeceras informativas sobre el estado del límite. Este comportamiento asegura una respuesta clara y coherente tanto desde el punto de vista técnico como funcional.

El sistema se ha diseñado de forma configurable, permitiendo definir distintos límites y ventanas temporales para cada endpoint, lo que facilita su adaptación a distintos escenarios de uso sin necesidad de modificar la lógica interna.

La integración de Redis se realizó mediante el docker-compose, manteniendo la coherencia con el entorno de despliegue local del proyecto. La validación del requisito se llevó a cabo mediante pruebas automatizadas con pytest, comprobando que el sistema devuelve correctamente un código 429 al superar los límites definidos en cada endpoint.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest tests/test_r14_rate_limit.py -v
platform win32 -- python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 3 items

tests/test_r14_rate_limit.py::test_p14_1_login_rate_limit_returns_429 PASSED [ 33%]
tests/test_r14_rate_limit.py::test_p14_1_query_rate_limit_returns_429 PASSED [ 66%]
tests/test_r14_rate_limit.py::test_p14_1_upload_rate_limit_returns_429 PASSED [100%]

3 passed in 5.85s
```

Figura 6.5 – Ejecución satisfactoria de las pruebas de rate limiting, verificando la devolución de HTTP 429 al superar los límites configurados en los endpoints críticos del sistema.

Este requisito introduce una medida adicional de seguridad operativa que complementa la autenticación definida en R11, el control de acceso desarrollado en R12 y la validación del bloque de seguridad realizada en R13.

Observabilidad y monitorización del sistema (R15)

En este requisito se han incorporado mecanismo de observabilidad con el objetivo de poder analizar el comportamiento interno del sistema, medir su rendimiento y facilitar la detección de errores durante su ejecución. A diferencia de los requisitos anteriores, este se plantea como un componente transversal que afecta a todo el backend.

En primer lugar, se ha implementado un sistema de logging estructurado en formato JSON. Cada petición registrada incluye información relevante como el identificador de la petición (*request_id*), el usuario asociado (*user_id*), la ruta solicitada, el método HTTP, el código de estado y el tiempo de respuesta en milisegundos. Esto permite disponer de trazas consistentes y fácilmente procesables, tanto en entornos de desarrollo como en producción.

Para generar esta información, se ha desarrollado un middleware que actúa sobre todas las peticiones entrantes. Este componente se encarga de asignar un identificados único a cada solicitud, medir el tiempo de ejecución completo y registrar los datos asociados una vez finalizada la respuesta. De este modo, se obtiene una visión completa del ciclo de vida de cada petición sin necesidad de modificar los endpoints de forma individual.

Además, se ha integrado un sistema de métricas compatible con Prometheus mediante la exposición del endpoint */metrics*. A través de este endpoint se pueden consultar tanto métricas generales del sistema como información específica sobre el rendimiento de la API, incluyendo el número total de peticiones y su duración por endpoint.

Como aprte más relevante de este requisito, se ha instrumentado el pipeline de consulta simulado (RAG) en tres etapas diferenciadas: *retrieval*, *embedding* y *llm*. Para cada una de ellas se registran tanto el número de ejecuciones como el tiempo de procesamiento, lo que permite analizar de forma detallada el comportamiento de cada fase y detectar posibles cuellos de botella.

Esta instrumentación se ha realizado de forma desacoplada mediante funciones auxiliares que encapsulan la medición de tiempos y el registro de métricas, evitando así

duplicar lógica en los endpoints y facilitando su reutilización en futuras ampliaciones del sistema.

```
[C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r15_observability.py
platform win32 -- python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 3 items

tests/test_r15_observability.py::test_p15_1_metrics_endpoint_is_available PASSED [ 33%]
tests/test_r15_observability.py::test_p15_1_request_id_header_is_returned PASSED [ 66%]
tests/test_r15_observability.py::test_p15_1_query_stage_metrics_are_exposed_after_query PASSED [100%]

3 passed in 0.93s
```

Figura 6.6 – Ejecución de pruebas de observabilidad. Salida de las pruebas automatizadas donde se verifica la correcta exposición del endpoint `/metrics` y la generación de métricas asociadas tanto a las peticiones HTTP como a las distintas etapas del pipeline de consulta.

```
1 # HELP archivum_http_requests_total Número total de peticiones HTTP procesadas por la API
2 # TYPE archivum_http_requests_total counter
3 archivum_http_requests_total{method="POST",path="/auth/login",status_code="200"} 2.0
4 archivum_http_requests_total{method="POST",path="/query",status_code="200"} 2.0
5
6 # HELP archivum_http_request_duration_seconds Duración de las peticiones HTTP en segundos
7 # TYPE archivum_http_request_duration_seconds histogram
8 archivum_http_request_duration_seconds_count{method="POST",path="/query"} 2.0
9 archivum_http_request_duration_seconds_sum{method="POST",path="/query"} 0.22002380003686994
10
11 # HELP archivum_rag_stage_total Número total de ejecuciones por etapa del pipeline RAG
12 # TYPE archivum_rag_stage_total counter
13 archivum_rag_stage_total{stage="retrieval",status="success"} 2.0
14 archivum_rag_stage_total{stage="embedding",status="success"} 2.0
15 archivum_rag_stage_total{stage="llm",status="success"} 2.0
16
17 # HELP archivum_rag_stage_duration_seconds Duración de las etapas del pipeline RAG en segundos
18 # TYPE archivum_rag_stage_duration_seconds histogram
19 archivum_rag_stage_duration_seconds_count{stage="retrieval"} 2.0
20 archivum_rag_stage_duration_seconds_count{stage="embedding"} 2.0
21 archivum_rag_stage_duration_seconds_count{stage="llm"} 2.0
```

Figura 6.7 – Métricas del sistema expuestas en `/metrics`. Visualización de las métricas generadas por el sistema en formato compatible con Prometheus, donde se observa el registro del número de ejecuciones y la duración de cada etapa del pipeline (retrieval, embedding y llm), junto con métricas de rendimiento de las peticiones HTTP.

Este requisito introduce capacidades de observabilidad que permiten monitorizar el comportamiento del sistema y analizar su rendimiento en tiempo real, complementando los mecanismos de seguridad implementados en R11, R12, R13 y R14, y facilitando el diagnóstico y evolución futura de la aplicación.

GESTIÓN Y PROCESAMIENTO DE DOCUMENTOS (R20 – R23)

Subida de documentos (R20)

En este requisito se ha implementado la funcionalidad de subida de documentos al sistema, permitiendo a los usuarios cargar archivos mediante peticiones HTTP en formato *multipart/form-data*. El objetivo principal ha sido habilitar un mecanismo sencillo pero seguro para almacenar archivos físicos en el servidor y registrar su información en la base de datos, manteniendo la asociación directa con el usuario autenticado.

La implementación se ha realizado mediante un endpoint específico (*/documents/upload*) que acepta tanto archivos como datos adicionales. En el caso de recibir un fichero, el sistema lo guarda físicamente en disco utilizando un nombre único generado automáticamente para evitar colisiones, mientras que se conserva el nombre original del archivo para su posterior referencia. Además, se registran metadatos relevantes como el tipo de MIME y el tamaño en bytes.

Por otro lado, se ha mantenido compatibilidad con el modo anterior de creación de documentos basado únicamente en texto, de forma que si no se proporciona archivo pero sí contenido textual, el sistema sigue funcionando sin romper requisitos previos ya implementados. Esto permite una transición progresiva entre ambos modelos sin afectar al resto del sistema.

En cuanto a la seguridad, la subida de documentos está protegida mediante autenticación JWT y control de acceso basado en roles. Solo los usuarios con permisos adecuados (por ejemplo, roles como admin o editor) pueden realizar esta acción, evitando así accesos no autorizados. Durante las pruebas se ha verificado también la correcta gestión de errores, como intentos de subida sin autenticación o sin los permisos necesarios.

A nivel de persistencia, cada documento queda asociado al usuario que lo ha subido mediante su identificador, garantizando la trazabilidad y permitiendo futuras funcionalidades como control de acceso, versionado o gestión de metadatos.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r20_document_upload.py
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 1 item

tests/test_r20_document_upload.py::test_p20_1_document_upload_success PASSED [100%]

===== 1 passed in 1.18s =====
```

Figura 6.8 – Ejecución satisfactoria de las pruebas de subida de documentos, verificando el almacenamiento correcto del archivo y su asociación al usuario autenticado.

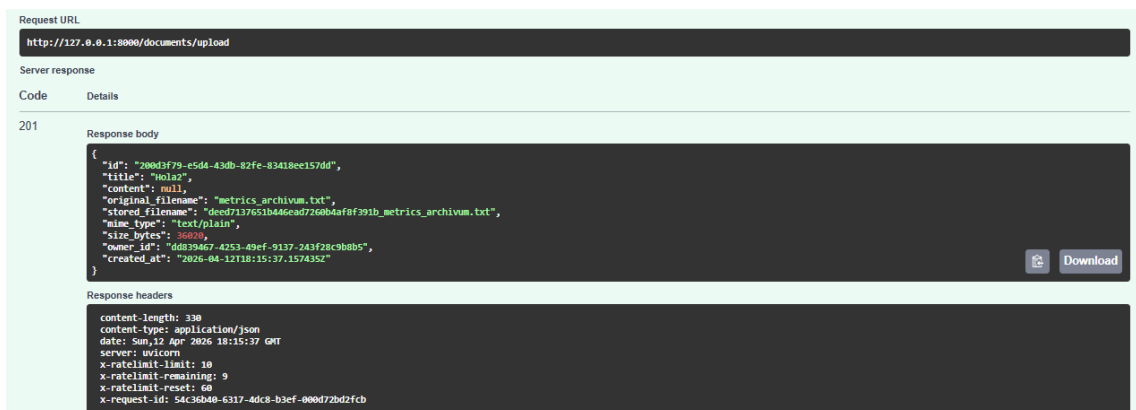


Figura 6.9 – Subida de un documento en formato .txt mediante Swagger, mostrando la respuesta del sistema con los metadatos generados (nombre original, nombre almacenado, tipo MIME y tamaño).

Este requisito establece la base necesaria para la incorporación de funcionalidades más avanzadas, como la gestión de metadatos, el versionado de documentos y la futura integración con procesos de análisis o ingesta de contenido.

Metadata de documentos (R21)

Una vez implementada la subida de documentos en el requisito anterior, se incorporó la gestión de metadata básica asociada a cada documento con el objetivo de facilitar su clasificación interna y preparar el sistema para futuras funcionalidades de recuperación y filtrado. Este requisito no se centra todavía en la búsqueda semántica ni en la aplicación de filtros avanzados, sino en establecer una base estructurada y persistente sobre la que puedan apoyarse capacidades posteriores.

Para ello, se definió un modelo específico de metadata en base de datos, vinculado directamente a la entidad de documento mediante una relación de asociación. La

solución adoptada sigue un esquema sencillo de pares clave-valor, de forma que cada documento puede almacenar varias entradas de metadata independientes, como categoría, idioma, área o cualquier otra etiqueta básica relevante para su organización. Este enfoque permite mantener una estructura flexible sin introducir una complejidad excesiva en esta fase del proyecto.

Desde el punto de vista funcional, se implementaron operaciones básicas de gestión que permiten registrar metadata sobre un documento ya existente, actualizar su valor cuando una clave ya está informada y consultar posteriormente la información asociada. De este modo, el sistema no solo conserva el archivo y sus datos físicos de subida, sino también una capa adicional de información estructurada útil para su clasificación. La persistencia de esta metadata en base de datos garantiza además su trazabilidad y reutilización en requisitos posteriores.

La lógica de gestión se integró dentro del módulo documental ya desarrollado, respetando la separación de responsabilidades definida en la arquitectura del sistema. La asociación entre documento y metadata se resolvió mediante clave foránea, garantizando integridad referencial y permitiendo recuperar de forma ordenada todas las entradas asociadas a un documento concreto. Asimismo, se incorporó una validación básica para evitar claves vacías y asegurar un comportamiento coherente en las operaciones de alta y actualización.

En cuanto al acceso, la gestión de metadata se apoya en las reglas de seguridad ya definidas previamente en el sistema. De esta forma, la consulta de metadata queda ligada a la posibilidad de acceso al documento, mientras que su modificación se restringe a usuarios autorizados, manteniendo coherencia con el modelo general de control de acceso aplicado al backend. Esto evita tratar la metadata como un elemento aislado y refuerza su integración dentro de la lógica documental completa.

La validación del requisito se realizó mediante pruebas automatizadas con pytest, verificando el ciclo completo de funcionamiento: creación de metadata para un documento, actualización de una clave existente, consulta del conjunto de metadatos asociados y comprobación de su persistencia en base de datos. También se comprobó el comportamiento ante intentos de modificación por parte de usuarios sin permisos suficientes, confirmando que las restricciones definidas se aplican de forma correcta.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r21_document_metadata.py
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 2 items

tests/test_r21_document_metadata.py::test_p21_1_document_metadata_management PASSED [ 50%]
tests/test_r21_document_metadata.py::test_p21_2_viewer_cannot_manage_metadata_of_foreign_document PASSED [100%]

===== 2 passed in 3.29s =====
```

Figura 6.10 – Ejecución satisfactoria de las pruebas de gestión de metadata documental, verificándola creación, actualización, consulta y persistencia de metadatos asociados a un documento.

Este requisito amplía la base de gestión documental iniciada en R20 y deja preparada la estructura necesaria para el uso posterior de la metadata en procesos de clasificación y filtrado dentro del sistema.

Versionado de documentos (R22)

Una vez implementadas la subida de documentos y la gestión de metadata en los requisitos anteriores, se incorporó un mecanismo de versionado documental con el objetivo de conservar el historial de cambios realizado sobre cada documento sin perder la referencia al estado actual del mismo. Este requisito resulta especialmente relevante en un sistema orientado a la gestión documental, ya que permite mantener trazabilidad sobre la evolución de la información almacenada y evitar la sobrescritura simple de contenido.

Para dar respuesta a esta necesidad, se definió un modelo específico de versión en base de datos, vinculado a la entidad documento mediante una relación de asociación. De este modo, cada documento puede disponer de varias versiones ordenadas secuencialmente, conservando para cada una de ellas la información necesaria para reconstruir su estado en un momento concreto. La solución adoptada permite almacenar tanto los datos propios del contenido como la información asociada al archivo subido en cada revisión, manteniendo una separación clara entre el documento como entidad lógica y sus distintas versiones registradas a lo largo del tiempo.

Desde el punto de vista funcional, se implementó la lógica necesaria para crear nuevas versiones sobre un documento ya existente, generar automáticamente la numeración correspondiente y conservar el historial completo de cambios. Además, se habilitó la consulta ordenada de versiones, de forma que el sistema puede recuperar tanto el listado histórico asociado a un documento como la información concreta de una versión

determinada. Con ello, el backend deja de tratar el documento como un recurso estático y pasa a gestionarlo como una entidad evolutiva, más coherente con un entorno real de trabajo documental.

A nivel de integración, la funcionalidad de versionado se incorporó dentro del módulo documental ya desarrollado, respetando la separación de responsabilidades definida previamente en la arquitectura general del sistema. La asociación entre documento y versión se resolvió mediante clave foránea, garantizando integridad referencial y permitiendo mantener la coherencia entre el estado actual del documento y su historial persistido. Esta decisión simplifica además la extensión futura del sistema, ya que el procesamiento posterior del contenido puede apoyarse sobre una versión concreta y no únicamente sobre el documento en abstracto. Esa continuidad queda alineada con la planificación del proyecto, donde R22 forma parte del núcleo documental y R30 asocia la extracción de texto a la versión del documento.

En cuanto al acceso, la gestión de versiones se apoya en las reglas de seguridad ya definidas para el sistema documental. De esta forma, la consulta del historial queda ligada al acceso autorizado al documento, mientras que la creación de nuevas versiones se restringe a los usuarios con permisos suficientes sobre dicho recurso. Con ello, el versionado no se trata como un mecanismo aislado, sino como una ampliación coherente del modelo de gestión y control de acceso ya implantado en requisitos anteriores.

La validación del requisito se realizó mediante pruebas automatizadas con pytest, verificando el ciclo básico de funcionamiento: creación de un documento inicial, generación de una nueva versión sobre el mismo, consulta del historial asociado y comprobación de la correcta persistencia de las versiones en base de datos. Asimismo, se comprobó que el estado actual del documento refleja la última versión registrada y que los usuarios sin permisos adecuados no pueden modificar el historial documental. De este modo, se confirmó que el sistema cumple con el objetivo planteado para este requisito: almacenar correctamente las versiones y ofrecer un historial accesible y trazable.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r22_document_versioning.py
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 2 items

tests/test_r22_document_versioning.py::test_p22_1_document_versioning PASSED [ 50%]
tests/test_r22_document_versioning.py::test_p22_2_viewer_cannot_create_new_version_of_foreign_document PASSED [100%]

===== 2 passed in 3.20s =====
```

Figura 6.11 – Ejecución satisfactoria de las pruebas de versionado documental, verificando la creación de nuevas versiones, su asociación correcta al documento y la consulta ordenada del historial almacenado.

Este requisito amplía la base de gestión documental iniciada en R20 y R21, incorporando trazabilidad sobre la evolución de los documentos y dejando preparada la estructura necesaria para su integración posterior con la ingesta y el procesamiento del contenido.

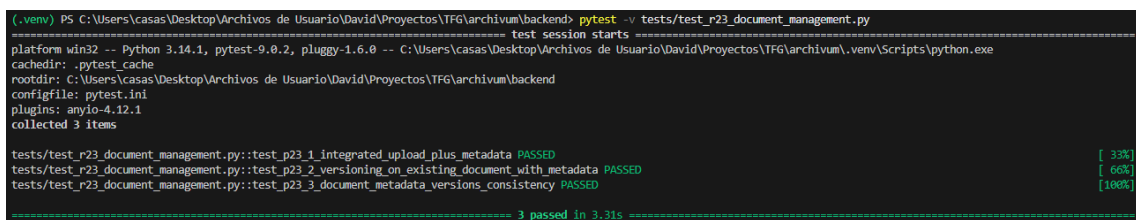
Tests de gestión documental (R23)

Una vez implementadas la subida de documentos, la gestión de metadata y el versionado en los requisitos anteriores, se llevó a cabo una fase específica de validación orientada a comprobar el funcionamiento conjunto del bloque documental del sistema. El objetivo de este requisito no fue repetir de forma aislada las pruebas ya realizadas en R20, R21 y R22, sino verificar que las tres funcionalidades operan de forma coherente cuando se encadenan dentro de un flujo real de uso. Esta orientación encaja con la definición de R23 dentro del proyecto, donde se establece expresamente una prueba integrada de subida y metadata, una prueba de versionado sobre documento existente y una prueba de coherencia entre documento, metadata y versiones.

La validación se estructuró en tres pruebas automatizadas con pytest. En primer lugar, se comprobó un flujo integrado en el que un usuario autenticado sube un documento al sistema y, sobre ese mismo recurso recién creado, registra varias entradas del metadata. Esta prueba permitió verificar no solo la creación correcta del documento y de sus metadatos asociados, sino también su recuperación posterior como un único conjunto coherente. En segundo lugar, se validó el versionado sobre un documento ya existente y previamente enriquecido con metadata, confirmando que la creación de una nueva versión no rompe la entidad documental lógica ni elimina la información ya asociada. Finalmente, se ejecutó una prueba de coherencia global en la que se contrastó que el documento actual refleja el estado de la última versión registrada, que la metadata sigue

vinculada al documento principal y que el historial conserva ordenadamente las versiones previas almacenadas. Este enfoque es coherente con la evolución funcional definida previamente en R21 y R22, donde la metadata se concibe como información asociada al documento y el versionado como un mecanismo de trazabilidad sobre su evolución.

Además de la validación a través de la API, en la prueba de coherencia final se contrastó la persistencia real en base de datos, revisando directamente la información almacenada en las tablas de documentos, metadata y versiones. De este modo, no solo se verificó que los endpoints devolvían respuestas correctas, sino también que estado persistido del sistema era consistente en dichas respuestas. Con ellos, R23 aporta una validación de integración más cercana al comportamiento real del sistema que las pruebas funcionales aisladas de los requisitos anteriores, y confirma que el bloque documental puede considerarse estable antes de continuar con requisitos posteriores de procesamiento e ingesta. Esta función de cierre del bloque documental también queda alineada con la planificación del proyecto, donde R23 aparece como requisito específico de pruebas dentro de la fase de gestión documental.



```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r23_document_management.py
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 3 items

tests/test_r23_document_management.py::test_p23_1_integrated_upload_plus_metadata PASSED [ 33%]
tests/test_r23_document_management.py::test_p23_2_versioning_on_existing_document_with_metadata PASSED [ 66%]
tests/test_r23_document_management.py::test_p23_3_document_metadata_versions_consistency PASSED [100%]

===== 3 passed in 3.31s =====
```

Figura 6.12 – Ejecución satisfactoria de las pruebas integradas de gestión documental, verificando la coherencia entre la subida de documentos, la asociación de metadata, la creación de nuevas versiones y el mantenimiento del historial documental.

Este requisito valida de forma conjunta el bloque de gestión documental del sistema, comprobando que las funcionalidades desarrolladas en R20, R21 y R22 no solo funcionan de manera aislada, sino también de forma coherente cuando se utilizan como parte de un mismo flujo documental.

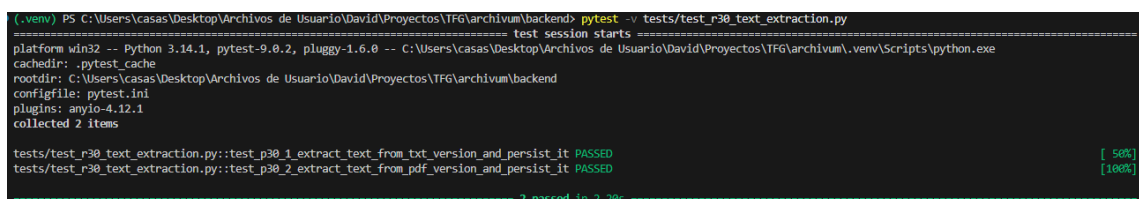
Extracción de texto (R30)

Una vez consolidada la base documental del sistema mediante la subida de archivos, la asociación de metadata y el versionado, se incorporó un mecanismo específico de extracción de texto asociado a cada versión del documento. Esta decisión mantiene la

coherencia con el diseño adoptado en requisitos anteriores, donde el documento se trata como una entidad lógica evolutiva y cada versión representa un estado concreto y persistente del mismo. De este modo, el procesamiento posterior no se aplica sobre el documento en abstracto, sino sobre una versión identificable y trazable, tal y como ya queda anticipado en la planificación del proyecto.

La solución implementada se apoya en un servicio de extracción desacoplado del almacenamiento físico y de la lógica de acceso HTTP. Este servicio recibe una versión documental y resuelve la estrategia de extracción en función de su origen. Si la versión fue creada directamente desde contenido textual, el texto se toma de dicho contenido; si la versión está asociada a un archivo, la extracción se realiza a partir del fichero almacenado en disco. En esta fase se ha dado soporte a formatos habituales dentro del alcance del proyecto, concretamente texto plano, Markdown, PDF y DOCX, dejando fuera otros escenarios más complejos que no forman parte de este requisito.

El texto extraído no se devuelve únicamente en memoria, sino que se persiste en la propia versión documental junto con información adicional de control, como el estado de la extracción, la fecha de procesamiento y, en caso de error, el mensaje asociado. Esta decisión reduce trabajo repetido en fases posteriores y deja preparada la base para los requisitos de chunking y pipeline asíncrono. La validación se realizó mediante pruebas automatizadas con pytest, comprobando tanto la extracción correcta desde archivos .txt como desde documentos PDF, así como la persistencia final del resultado en base de datos.



```
(.venv) PS C:\Users\Casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r30_text_extraction.py
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest cache
rootdir: C:\Users\Casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 2 items

tests/test_r30_text_extraction.py::test_p30_1_extract_text_from_txt_version_and_persist_it PASSED [ 50%]
tests/test_r30_text_extraction.py::test_p30_2_extract_text_from_pdf_version_and_persist_it PASSED [100%]

===== 2 passed in 2.20s =====
```

Figura 6.13 – Ejecución satisfactoria de las pruebas de extracción de texto, verificando la recuperación del contenido desde versiones documentales en formato texto plano y PDF, así como su persistencia correcta en la base de datos.

Este requisito amplía la fase de ingesta iniciada tras el versionado documental y deja preparada la base necesaria para abordar el chunking y la orquestación posterior del pipeline en los requisitos R31 y R32.

Chunking (R31)

Una vez resuelta la extracción de texto en el requisito anterior, se incorporó la fase de fragmentación del contenido textual con el objetivo de transformar el texto completo de cada versión documental en unidades más pequeñas y manejables. Este paso resulta necesario para preparar la información de cara a las fases posteriores de vectorización y recuperación semántica, ya que trabajar directamente sobre textos largos y continuos dificultaría tanto el tratamiento técnico como la precisión de las búsquedas. La propia planificación del proyecto sitúa este requisito inmediatamente después de la extracción y antes del pipeline asíncrono y la generación de embeddings, lo que refuerza su papel como fase intermedia dentro del bloque de ingesta y procesamiento.

Para dar respuesta a esta necesidad, se implementó una lógica de chunking desacoplada del endpoint HTTP y de la persistencia, de manera que la fragmentación pudiera tratarse como una responsabilidad concreta dentro del módulo de procesamiento documental. La solución adoptada parte del texto ya extraído y asociado a una versión específica del documento, evitando operar sobre el documento en abstracto y manteniendo así la trazabilidad ya introducida en los resultados de versionado y extracción. Esta continuidad es coherente con la base ya descrita en la memoria, donde el texto extraído se persiste en la versión documental precisamente para facilitar fases posteriores como el chunking.

La estrategia elegida para esta fase se ha basado en una fragmentación secuencial del texto en bloques de tamaño controlado, aplicando un criterio simple y justificable para el alcance académico del proyecto. En lugar de incorporar técnicas avanzadas de segmentación semántica o ajuste dinámico por estructura documental, se optó por un enfoque básico apoyado en el tamaño del fragmento y en su posición dentro del texto original. Esta decisión permite mantener una complejidad contenida y, al mismo tiempo, generar unidades de información suficientemente pequeñas como para ser reutilizadas en la etapa de embeddings sin perder completamente el contexto local del contenido. En este sentido, el requisito se ajusta de forma directa a su definición inicial, centrada en la división del texto en chunks, el cálculo básico de tamaño y la persistencia asociada, excluyendo tanto la generación de embeddings como cualquier optimización avanzada del proceso.

Cada fragmento generado se almacena como una entidad propia vinculada tanto al documento como a la versión concreta desde la que procede. Además del contenido textual del chunk, se conserva información básica de control, como su posición ordinal dentro de la secuencia y su tamaño, lo que permite reconstruir el orden lógico del texto original y mantener trazabilidad sobre el procesamiento realizado. Esta decisión resulta especialmente importante de cara a requisitos posteriores, ya que la vectorización y la búsqueda semántica no trabajarán sobre documentos completos, sino sobre estos fragmentos persistidos como unidades independientes. La arquitectura definida en la memoria ya anticipa este enfoque al incluir de forma explícita las entidades Chunk y Embedding dentro del modelo de datos y al situar la fragmentación como parte del módulo de procesamiento documental.

A nivel de integración, la funcionalidad de chunking se incorporó respetando la separación de responsabilidades del backend. La generación de fragmentos parte exclusivamente del texto extraído en R30 y deja preparado el resultado para el pipeline asíncrono descrito en R32 y para la generación de embeddings de R40. De este modo, R31 no se plantea como una funcionalidad aislada, sino como una pieza de continuidad dentro del flujo completo de procesamiento documental. Esta relación entre requisitos también aparece recogida en la planificación y en la descomposición RFTP del proyecto, donde R31 se estructura mediante la función de fragmentación del texto, la lógica de chunking, la persistencia de chunks y su prueba específica.

La validación del requisito se realizó mediante una prueba automatizada con pytest, verificando que el texto previamente extraído desde una versión documental se divide correctamente en varios fragmentos, que dichos fragmentos conservan el orden esperado y que todos queda almacenados en base de datos asociados a la versión correspondiente. Además, la prueba permite comprobar que la persistencia del resultado no se limita a la respuesta del endpoint, sino que queda registrada de forma consistente en la estructura documental del sistema. Con ello, se confirma el cumplimiento de la definición de done establecida para este requisito: el texto queda fragmentado correctamente, los chunks se asocian a su documento y la funcionalidad queda validada mediante pruebas satisfactorias.

Este requisito amplía la fase de ingesta iniciada con la extracción de texto y deja preparada la estructura necesaria para la orquestación del pipeline de procesamiento y la posterior generación de embeddings a partir de fragmentos persistidos y trazables.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r31_chunking.py
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 1 item

tests/test_r31_chunking.py::test_p31_1_chunk_text_and_persist_chunks_correctly PASSED [100%]

===== 1 passed in 1.24s =====
```

Figura 6.14 – Ejecución satisfactoria de las pruebas de chunking, verificando la fragmentación correcta del texto extraído, el mantenimiento del orden de los fragmentos y su persistencia asociada a la versión documental correspondiente.

Pipeline asíncrono (R32)

En este requisito se implementa un pipeline asíncrono encargado de coordinar las fases de procesamiento de los documentos dentro del sistema. Concretamente, este pipeline se encarga de orquestar la extracción de texto (R30) y el posterior proceso de fragmentación en chunks (R31), dejando la información preparada para la fase de vectorización definida en requisitos posteriores.

Para ello se ha diseñado un mecanismo basado en la ejecución en segundo plano mediante *BackgroundTask* de FastAPI, lo que permite lanzar el procesamiento sin bloquear la petición HTTP inicial. De esta forma, el usuario puede iniciar el pipeline y continuar trabajando mientras el sistema realiza las tareas de forma asíncrona.

El proceso se articula a través de una entidad específica denominada *PipelineJob*, que representa cada ejecución del pipeline sobre una versión concreta de un documento. Esta entidad permite almacenar el estado del proceso, el paso actual en el que se encuentra y posibles errores producidos durante la ejecución. Los estados definidos son *pending*, *running*, *completed* y *failed*, mientras que los pasos internos incluyen *extracting_text*, *chunking_test* y *ready_for_vectorization*.

El flujo de ejecución del pipeline sigue un orden secuencial bien definido. En primer lugar, se crea un job en estado pendiente asociado a una versión del documento. A continuación, se inicia la extracción del texto, persistiendo el resultado en la base de datos. Posteriormente, se realiza el proceso de chunking utilizando los parámetros definidos (tamaño de fragmento y solapamiento), generando y almacenando los

fragmentos correspondientes. Finalmente, el sistema marca la versión como lista para la siguiente fase del procesamiento.

En cuanto a la gestión de errores, se ha optado por un enfoque sencillo pero efectivo. Cualquier fallo durante la ejecución provoca la transición del job al estado *failed*, registrando un mensaje descriptivo que facilita su análisis posterior. No se han implementado mecanismos avanzados de reintento, ya que quedan fuera del alcance de este requisito.

A nivel de acceso, únicamente los usuarios con permisos adecuados (propietario del documento o administrador) pueden lanzar el pipeline, mientras que la consulta del estado del job está disponible para usuarios con acceso al documento.

Para la validación del requisito, se han desarrollado pruebas que verifican tanto la ejecución completa del pipeline como el manejo de errores. Estas pruebas comprueban que el flujo se ejecuta en el orden correcto, que los datos se persisten adecuadamente y que los estados del job reflejan correctamente la evolución del proceso.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r32_async_pipeline.py
test session starts
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 2 items

tests/test_r32_async_pipeline.py::test_p32_1_run_pipeline_and_complete_all_steps PASSED [ 50%]
tests/test_r32_async_pipeline.py::test_p32_2_mark_job_as_failed_when_extraction_fails PASSED [100%]

===== 2 passed in 2.37s =====
```

Figura 6.15 – Ejecución satisfactoria del pipeline asíncrono, mostrando la transición de estados desde *pendieng* hasta *completed* y la generación correcta de los chunks asociados al documento.

En conjunto, este requisito introduce la base del procesamiento desacoplado del sistema, permitiendo evolucionar hacia arquitecturas más escalables en fases posteriores del proyecto.

Tests del pipeline (R33)

Una vez implementadas la extracción de texto, la fragmentación en chunks y la ejecución asíncrona del pipeline en los requisitos anteriores, se llevó a cabo una fase específica de validación orientada a comprobar el funcionamiento conjunto de todo el bloque de ingesta y procesamiento documental. El objetivo de este requisito no fue repetir de forma aislada las pruebas de R30, R31 y R32, sino verificar que el pipeline completo se comporta de forma coherente cuando se ejecuta sobre documentos reales y persiste sus resultados en el sistema.

La validación se estructuró en tres pruebas automáticas con pytest. En primer lugar, se definió una prueba end-to-end en la que un usuario autenticado sube un documento en formato de texto, lanza el pipeline asíncrono sobre su primera versión y espera a la finalización del job. Esta prueba permitió comprobar de forma conjunta la extracción del contenido, la generación de chunks y la transición del proceso hasta el estado **completed**, verificando además que la versión queda marcada como lista para la fase posterior de vectorización.

En segundo lugar, se validó la coherencia de estados del pipeline tanto en un escenario correcto como en un caso de error controlado. Para ello, se contrastó que un documento soportado evoluciona por los estados esperados hasta completarse correctamente, mientras que un archivo en formato no soportado provoca el paso del job a estado **failed**, registrando además el mensaje de error correspondiente. Con ello, no solo se comprobó el flujo nominal, sino también la respuesta del sistema ante una situación de fallo previsible.

Por último, se incorporó una prueba centrada en la coherencia entre el texto extraído, los chunks persistidos y el estado final del job. Esta validación permitió verificar que cada fragmento almacenado corresponde exactamente al tramo de texto delimitado por sus posiciones de inicio y fin, que la secuencia de chunks mantiene el orden esperado y que el número de fragmentos coincide con la información registrada por el pipeline. De este modo, R33 no se limita a confirmar que el proceso termina, sino que verifica que el resultado persistido es consistente y trazable.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -v tests/test_r33_pipeline_tests.py
===== test session starts =====
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 3 items

tests/test_r33_pipeline_tests.py::test_p33_1_end_to_end_pipeline_processes_document_completely PASSED [ 33%]
tests/test_r33_pipeline_tests.py::test_p33_2_pipeline_state_transitions_are_consistent_for_success_and_failure PASSED [ 66%]
tests/test_r33_pipeline_tests.py::test_p33_3_extracted_text_chunks_and_job_state_remain_consistent PASSED [100%]

===== 3 passed in 4.87s =====
```

Figura 6.16 – Ejecución satisfactoria de las pruebas del pipeline, verificando el procesamiento completo del documento, la coherencia de estados del job y la consistencia entre texto extraído y chunks persistidos.

Este requisito valida de forma conjunta el bloque de ingesta y procesamiento del sistema, confirmando que las funcionalidades desarrolladas en R30, R31 y R32 no solo

funcionan por separado, sino también de forma coherente dentro del flujo completo del pipeline.

Generación de embeddings (R40)

Una vez completadas las fases de extracción de texto (R30), fragmentación en chunks (R31) y orquestación del pipeline documental (R32), se desarrolló el requisito R40, centrado en la generación de embeddings a partir de los fragmentos textuales obtenidos. Este proceso constituye la base técnica de la búsqueda semántica, ya que permite representar matemáticamente el contenido textual en forma de vectores numéricos comparables entre sí.

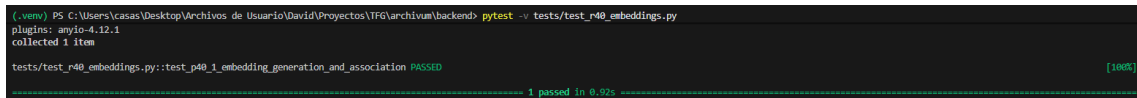
Para ello se implementó un servicio específico encargado de recorrer los chunks asociados a una versión documental concreta y enviar su contenido a un modelo de embeddings predefinido. En esta fase se optó por una integración desacoplada mediante cliente HTTP hacia un proveedor externo, permitiendo mantener la arquitectura abierta ante futuros cambios de modelo o proveedor sin rehacer la lógica interna del sistema. Cada fragmento procesado devuelve un vector numérico que conserva información semántica del texto original.

El sistema registra cada embedding generado y lo asocia de forma individual al chunk correspondiente, manteniendo la trazabilidad completa entre documento, versión, fragmento y representación vectorial. Para ello se incorporó una nueva entidad persistente orientada a almacenar el modelo utilizado, número de dimensiones, fecha de generación y resultado devuelto por el proveedor. Aunque en este requisito el almacenamiento se realiza en formato estructurado estándar, la transición a soporte vectorial nativo se reserva para el requisito R41.

Desde el punto de vista funcional, se habilitó un endpoint protegido que permite lanzar la generación de embeddings sobre una versión concreta del documento, reutilizando los controles de autenticación y permisos ya implantados en requisitos anteriores. De este modo, únicamente usuarios autorizados pueden iniciar procesos de vectorización sobre documentos propios o administrados.

En el apartado de validación, se diseñó una prueba automatizada que verifica el flujo completo: creación del documento, extracción del texto, chunking previo, generación de

embeddings y persistencia final. Para evitar dependencias externas durante la ejecución de tests, se simuló la respuesta del proveedor mediante mocks controlados, técnica habitual en entornos profesionales.



```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\IFG\archivos\backend> pytest -v tests/test_r40_embeddings.py
plugins: anyio-4.12.1
collected 1 item

tests/test_r40_embeddings.py::test_r40_1_embedding_generation_and_association PASSED [100%]

..... 1 passed in 0.52s
```

Figura 6.17 – Ejecución satisfactoria de la prueba automatizada del requisito R40, verificando la generación correcta de embeddings y su asociación con los chunks del documento procesado.

Con la finalización de este requisito, el sistema dispone ya de representaciones semánticas listas para su almacenamiento vectorial y posterior explotación en búsquedas inteligentes, consolidando el paso entre procesamiento documental clásico y recuperación avanzada de información.

Almacenamiento pgvector (R41)

Una vez implementada la generación de embeddings en el requisito anterior, el siguiente paso consistió en definir un mecanismo adecuado para su persistencia en base de datos. Para ello se incorporó la extensión pgvector sobre PostgreSQL, lo que permite almacenar vectores numéricos de forma nativa y preparar el sistema para futuras búsquedas por similitud semántica.

La solución desarrollada sustituyó el almacenamiento previo en formato JSON por una columna vectorial real dentro de la tabla de embeddings. De este modo, cada fragmento de documento (chunk) mantiene asociado su embedding correspondiente mediante una estructura optimizada y compatible con operaciones vectoriales posteriores. Esta decisión mejora la coherencia del modelo de datos y evita tratar los embeddings como simples listas de números sin semántica propia.

A nivel técnico, fue necesario adaptar el modelo ORM del backend para mapear el nuevo tipo de dato vectorial, registrar la compatibilidad con el driver de conexión y generar una migración de base de datos que activase la extensión vector y transformase la estructura existente. También se actualizaron las pruebas automatizadas para validar el nuevo comportamiento, comprobando tanto la persistencia correcta como la dimensión esperada de los vectores generados.

Durante la implantación se detectó la existencia de embeddings antiguos utilizados en pruebas previas con dimensiones reducidas, por lo que fue necesario limpiar dichos datos antes de ejecutar la migración definitiva. Tras ello, el sistema quedó funcionando de forma estable con embeddings normalizados de 1536 dimensiones, alineados con el modelo configurado.

Como resultado, Archivum dispone ahora de una base sólida para requisitos posteriores relacionados con indexación vectorial y búsqueda semántica eficiente, manteniendo además la integración con el pipeline documental desarrollado en fases anteriores.

```

C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivos\backend> pytest -v tests/test_r40_embeddings.py tests/test_r41_pgvector_storage.py
test session starts
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivos\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivos\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 2 items

tests/test_r40_embeddings.py::test_p40_1_generate_embeddings PASSED [ 50%]
tests/test_r41_pgvector_storage.py::test_p41_1_pgvector_storage PASSED [100%]

2 passed in 2.82s

```

Figura 6.18 – Ejecución satisfactoria de las pruebas automáticas del requisito R41 sobre almacenamiento vectorial con pgvector.

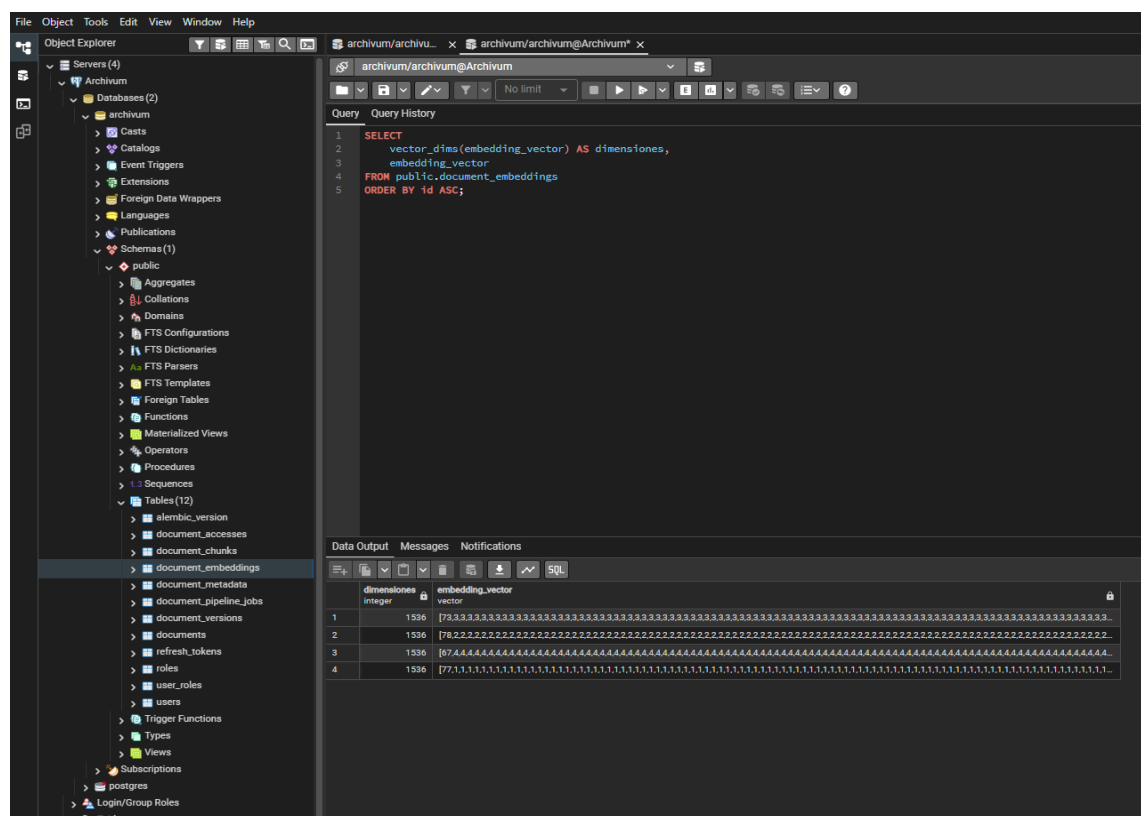


Figura 6.19 – Verificación en base de datos de la columna embedding_vector como tipo vector dentro de PostgreSQL.

Con la finalización de este requisito, el sistema deja preparada la capa de persistencia semántica necesaria para evolucionar hacia consultas vectoriales reales.

Índices vectoriales (R42)

Una vez completa el almacenamiento de embeddings en PostgreSQL mediante pgvector en el requisito anterior, el siguiente paso consistió en preparar la base de datos para realizar consultas vectoriales de forma más eficiente. Para ello se implementó un índice específico sobre la columna *embedding_vector*, donde se almacenan los vectores generados para cada fragmento documental.

La solución adoptada fue la creación de un índice de tipo *ivfflat*, proporcionado por la extensión pgvector. Este tipo de índice está orientado a operaciones de búsqueda por similitud entre vectores y permite reducir el coste de las consultas cuando el volumen de embeddings aumenta. En esta fase no se realizaron ajustes avanzados de rendimiento, ya que el objetivo principal era dejar preparada la infraestructura base para los requisitos posteriores de búsqueda semántica.

Como métrica principal de comparación se configuró la distancia por coseno mediante *vector_cosine_ops*, una opción habitual en sistemas de embeddings al centrarse en la similitud direccional entre vectores más que en su magnitud absoluta. Esta elección resulta adecuada para modelos de lenguaje y escenarios de recuperación semántica.

La creación del índice se integró mediante una nueva migración de Alembic, lo que permite reproducir el cambio de forma controlada en cualquier entorno. Además, tras la creación se ejecutó un análisis de estadísticas sobre la tabla para que PostgreSQL dispusiera de información actualizada al planificar consultas.

A nivel de backend también se añadió una consulta vectorial básica dentro del repositorio de embeddings. Esta operación recibe un vector de entrada y devuelve los fragmentos más cercanos ordenados por similitud. Aunque todavía no representa el buscador semántico completo, sí valida que toda la cadena técnica funciona correctamente: vectores generados, vectores almacenados, índice activo y consulta operativa.

Para verificar el requisito, se desarrolló una prueba automatizada que comprueba la existencia del índice en PostgreSQL, su configuración con *ivfflat* y *vector_cosine_ops*, y la recuperación correcta del chunk más similar al vector consultado.

Con esta implementación queda preparada la base técnica necesaria para abordar los requisitos de búsqueda, semántica y búsqueda híbrida definidos en fases posteriores.

```
(.venv) PS C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend> pytest -- tests/test_r42_vector_indexes.py
platform win32 -- Python 3.14.1, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\casas\Desktop\Archivos de Usuario\David\Proyectos\TFG\archivum\backend
configfile: pytest.ini
plugins: anyio-4.12.1
collected 1 item

tests/test_r42_vector_indexes.py::test_p42_1_vector_index_and_similarity_query PASSED [100%]

1 passed in 1.34s
```

Figura 6.20 – Ejecución satisfactoria del test R42 y verificación del índice vectorial configurado sobre la tabla de embeddings.

Query

Query History

1

EXPLAIN ANALYZE

2

SELECT

3

de.chunk_id

4

FROM document_embeddings de

5

ORDER BY de.embedding_vector <=> (

6

SELECT embedding_vector

7

FROM document_embeddings

8

LIMIT 1

9

)

10

LIMIT 5;

Data Output

Messages

Notifications

QUERY PLAN

text

1

Limit (cost=10000000001.26..10000000524.34 rows=4 width=24) (actual time=395.546..395.550 rows=1 loops=1)

2

InitPlan 1 (returns \$0)

3

-> Limit (cost=10000000000.00..10000000000.26 rows=1 width=18) (actual time=0.029..0.030 rows=1 loops=1)

4

-> Seq Scan on document_embeddings (cost=10000000000.00..10000000001.04 rows=4 width=18) (actual time=0.023..0.024 rows=1 loops=1)

5

-> Index Scan using ix_document_embeddings_embedding_vector_cosine on document_embeddings de (cost=1.00..524.08 rows=4 width=24) (actual time=4.260..4.262 rows=1 lo...)

6

Order By: (embedding_vector <=> \$0)

7

Planning Time: 0.555 ms

8

JIT:

9

Functions: 9

10

Options: Inlining true, Optimization true, Expressions true, Deforming true

11

Timing: Generation 3.528 ms, Inlining 179.329 ms, Optimization 112.317 ms, Emission 99.666 ms, Total 394.840 ms

12

Execution Time: 1003.051 ms

Total rows: 12

Query complete 00:00:01.003

Figura 6.21 – Plan de ejecución de una consulta vectorial en PostgreSQL donde se utiliza el índice ix_document_embeddings_embedding_vector_cosine para recuperar embeddings por similitud mediante pgvector.